

Files

1 Basic Idea

Definition

A **file** is an operating-system abstraction for storing information on secondary storage and reading it back later.

- A file hides the physical details of disks and storage devices.
- Users do not need to know where the data are physically stored.
- Users do not need to know how the disk hardware works.
- A process creates a file and gives it a name.
- The file can continue to exist after the process that created it terminates.
- Other processes can later access the file by using its name.

Core Point

The file abstraction separates the user's view of stored information from the hardware details of storage.

2 File Naming

Why Naming Matters

Naming is one of the most important parts of any abstraction because names are how users and processes refer to stored objects.

- File-naming rules vary between operating systems.
- Most operating systems allow alphabetic strings as file names.
- Examples: `andrea`, `bruce`, `cathy`.
- Many systems also allow digits and special characters.
- Examples: `2`, `urgent!`, `Fig.2-14`.
- Many modern file systems support file names up to about 255 characters.

3 Case Sensitivity

Case Sensitivity

A file system is **case-sensitive** if uppercase and lowercase letters are treated as different characters in file names.

System type	Meaning	Example
Case-sensitive	Uppercase and lowercase names can refer to different files.	UNIX can treat <code>maria</code> , <code>Maria</code> , and <code>MARIA</code> as three distinct files.
Case-insensitive	Uppercase and lowercase versions of a name refer to the same file.	MS-DOS treats <code>maria</code> , <code>Maria</code> , and <code>MARIA</code> as the same file.

Common Mistake

Do not assume that file names behave the same way on every operating system. Case sensitivity depends on the file system and OS rules.

4 Windows File-System Note

- Older MS-DOS used the FAT file system.
- Windows 95 and Windows 98 used FAT-16 and later FAT-32 extensions.
- Newer Windows versions support FAT variants for compatibility.
- Newer Windows systems also use NTFS as a more advanced native file system.
- NTFS supports features not found in old FAT systems, such as richer metadata and Unicode file names.
- exFAT is a FAT-like Microsoft file system designed for flash drives and large file systems.

5 File Extensions

Definition

A **file extension** is the part of a file name after a period, often used to indicate the file's type or intended use.

- Many systems support two-part file names such as `prog.c`.
- The part before the period is the main name.
- The part after the period is the extension.
- MS-DOS used names of 1 to 8 characters plus an optional 1 to 3 character extension.
- UNIX allows extensions by convention and does not restrict them to three characters.
- UNIX file names may contain multiple extensions, such as `homepage.html.zip`.

6 Common Extensions

Extension	Meaning
<code>.bak</code>	Backup file.
<code>.c</code>	C source program.
<code>.gif</code>	GIF image file.
<code>.hlp</code>	Help file.
<code>.html</code>	HTML Web document.
<code>.jpg</code>	JPEG still picture.
<code>.mp3</code>	MPEG layer 3 audio file.
<code>.mpg</code>	MPEG movie file.
<code>.o</code>	Object file produced by a compiler, not yet linked.
<code>.pdf</code>	Portable Document Format file.
<code>.ps</code>	PostScript file.
<code>.tex</code>	Input file for the $\text{\TeX}$ formatting system.
<code>.txt</code>	General text file.
<code>.zip</code>	Compressed archive.

7 Extensions as Conventions

- In UNIX-like systems, extensions are usually conventions.
- The operating system generally does not enforce the meaning of an extension.
- A file named `file.txt` is expected to be text, but the OS does not guarantee it.
- Programs may still enforce their own extension rules.
- A C compiler may require source files to end in `.c`.
- Extensions help programs distinguish different input file types.

Compiler Example

A compiler can use extensions to distinguish C source files, assembly-language files, object files, and other input files.

8 Extensions with OS Meaning

- Windows assigns more direct meaning to file extensions.
- Users or programs can register an extension with the operating system.
- The registration says which application owns that extension.
- When the user double-clicks a file, Windows can launch the associated application.
- Example: double-clicking `file.docx` opens Microsoft Word with that file.

9 UNIX vs. Windows View

Feature	UNIX-like systems	Windows-style systems
Case handling	Often case-sensitive.	Often case-insensitive for common user file systems.
Extension meaning	Mostly a convention for users and programs.	Often registered with the OS and tied to applications.
Multiple extensions	Common and allowed, such as <code>file.html.zip</code> .	Allowed, but the last extension often drives association.
OS enforcement	OS usually does not care whether content matches extension.	OS often uses the extension to choose the default application.

Final Point

Files give users a named, persistent storage abstraction. File-name rules, case sensitivity, and extension meaning depend on the operating system and file system.

File Structure

1 Basic Idea

Definition

File structure describes how the contents of a file are organized and what meaning the operating system gives to that organization.

- A file can be viewed in several different ways.
- The file structure determines what the operating system understands about the file.
- Some systems treat files as raw byte sequences.
- Other systems treat files as records or indexed trees.
- The more structure the OS understands, the more help it can provide.
- The less structure the OS enforces, the more flexible files become for user programs.

2 Three Common File Structures

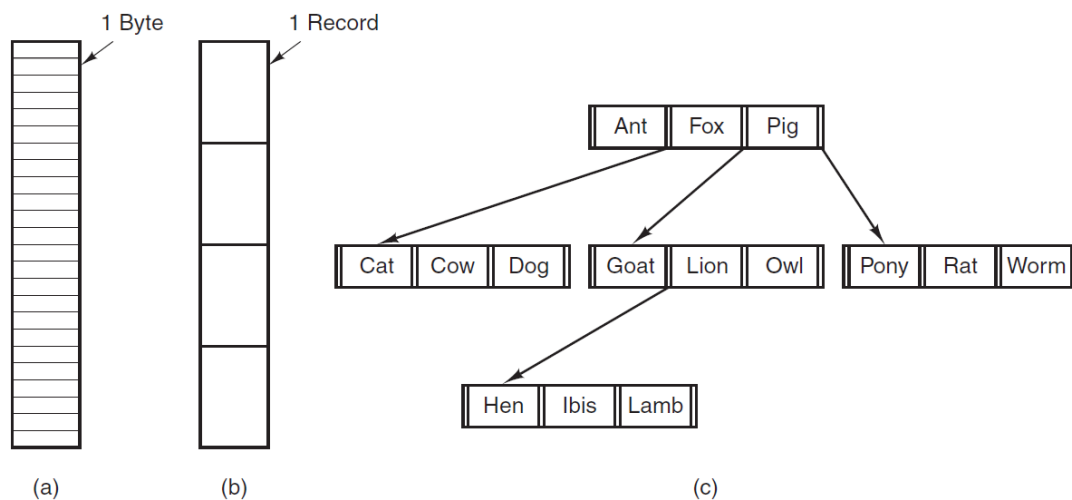


Figure 4-2. Three kinds of files. (a) Byte sequence. (b) Record sequence. (c) Tree.

Structure	Main idea
Byte sequence	The file is an unstructured sequence of bytes.
Record sequence	The file is a sequence of fixed-length records.
Tree	The file is a tree of records sorted by a key field.

3 Byte Sequence Files

Byte Sequence

A **byte sequence file** is a file whose contents are treated by the operating system as a simple sequence of bytes.

- The operating system does not know what the bytes mean.
- The operating system does not enforce records, fields, or keys.
- Any meaning is imposed by user-level programs.
- A text editor may interpret bytes as characters.
- A compiler may interpret bytes as source code.
- An image viewer may interpret bytes as encoded image data.

Used By

UNIX, Linux, OS X, and Windows use the byte-sequence file model.

4 Why Byte Sequences Are Flexible

- Programs can store any format they want.
- The OS does not need to understand every possible file type.
- New application file formats can be created without changing the OS.
- The OS does not help interpret file contents.
- The OS also does not get in the way of unusual file formats.

Trade-Off

The byte-sequence model gives maximum flexibility, but the responsibility for interpreting the file belongs to user programs.

5 Record Sequence Files

Record Sequence

A **record sequence file** is a file made of fixed-length records, where each record has some internal structure.

- The file is read and written in record units.
- A read operation returns one record.

- A write operation overwrites or appends one record.
- Records are usually the same size.
- This model was common on older mainframe systems.

6 Historical Record Examples

- Older systems often used 80-character records.
- This matched 80-column punched cards.
- Some systems also supported 132-character records.
- This matched old 132-column line printers.
- Programs read input in 80-character units.
- Programs wrote output in 132-character units.
- Current general-purpose systems rarely use this as their main file model.

Historical Note

Record-sequence files matched the physical media and printers used by older mainframe systems.

7 Tree-Structured Files

Tree File

A **tree-structured file** stores records in a tree, usually sorted by a key field in each record.

- Records do not have to be the same length.
- Each record contains a key field.
- The key field is located at a fixed position in each record.
- The tree is sorted by the key.
- This allows fast searching by key.
- The main operation is often lookup by key, not reading the next record.

Example

In a tree-structured zoo file, the system could directly search for the record with key pony without the user knowing its physical position.

8 Adding Records in Tree Files

- New records can be added by key.
- The operating system decides where the new record should be placed.
- The user does not manually choose the physical position.
- This is very different from the byte-sequence model.
- Tree-structured files have been used on large mainframe systems for commercial data processing.

9 Comparison

File structure	Strength	Weakness
Byte sequence	Maximum flexibility; programs can define any format.	OS gives no help interpreting file contents.
Record sequence	Simple record-based read and write operations.	Inflexible for data that does not naturally fit fixed-size records.
Tree	Fast lookup by key and OS-managed placement.	More complex and less general than byte streams.

10 Key Contrast

- Byte-sequence files are application-defined.
- Record-sequence files are operation-defined around fixed records.
- Tree-structured files are search-defined around keys.
- Modern general-purpose systems prefer byte sequences because they are simple and flexible.
- Specialized systems may use records or trees when the OS needs to support structured access directly.

Final Point

The more structure the operating system imposes on files, the more services it can provide, but the less freedom applications have to define their own formats.

File Types

1 Basic Idea

Definition

File type describes the role or interpretation of a file, such as regular file, directory, special device file, ASCII text file, or binary file.

- Many operating systems support more than one kind of file.
- UNIX-like systems and Windows support regular files and directories.
- UNIX also supports character special files and block special files.
- This note mainly focuses on regular files.
- Regular files contain user information.

2 Major File Categories

File type	Meaning
Regular file	Contains ordinary user information, such as text, programs, images, or data.
Directory	System file used to maintain the structure of the file system.
Character special file	Models serial I/O devices such as terminals, printers, and networks.
Block special file	Models block devices such as disks.

Core Point

Regular files store user data, while directories and special files help the operating system organize names and model devices.

3 Regular Files

Regular File

A **regular file** is a file that contains user information rather than directory metadata or device access information.

- Regular files may contain text.
- They may contain executable programs.
- They may contain images, audio, video, archives, or application data.
- The operating system may treat the file as a byte sequence.

- Programs usually interpret the actual contents.
- Regular files are commonly divided into ASCII files and binary files.

4 ASCII Files

ASCII File

An **ASCII file** is a regular file made of lines of text.

- ASCII files contain printable text characters.
- Lines do not have to be the same length.
- Some systems terminate lines with a carriage return.
- Some systems terminate lines with a line feed.
- Windows traditionally uses both carriage return and line feed.
- ASCII files can be displayed, printed, and edited easily.

Main Advantage

ASCII files are portable and easy for both humans and programs to process.

5 Why ASCII Files Are Useful

- They can be edited with any text editor.
- They can be printed directly.
- They are easy to inspect for debugging.
- They work well as input and output for many programs.
- Shell pipelines are easier to use when programs exchange text.
- Standard text conventions make program output easier for another program to interpret.

6 Binary Files

Binary File

A **binary file** is a regular file that is not an ASCII text file and usually has an internal structure understood by specific programs.

- Binary files may contain arbitrary byte values.
- Printing a binary file as text usually produces unreadable output.

- Binary files often contain numeric fields, headers, offsets, tables, or encoded data.
- The operating system may recognize some binary formats.
- Programs that use the file know how to interpret its internal structure.

Do Not Print Blindly

Sending a binary file directly to a printer or terminal may produce meaningless characters.

7 Binary File Examples

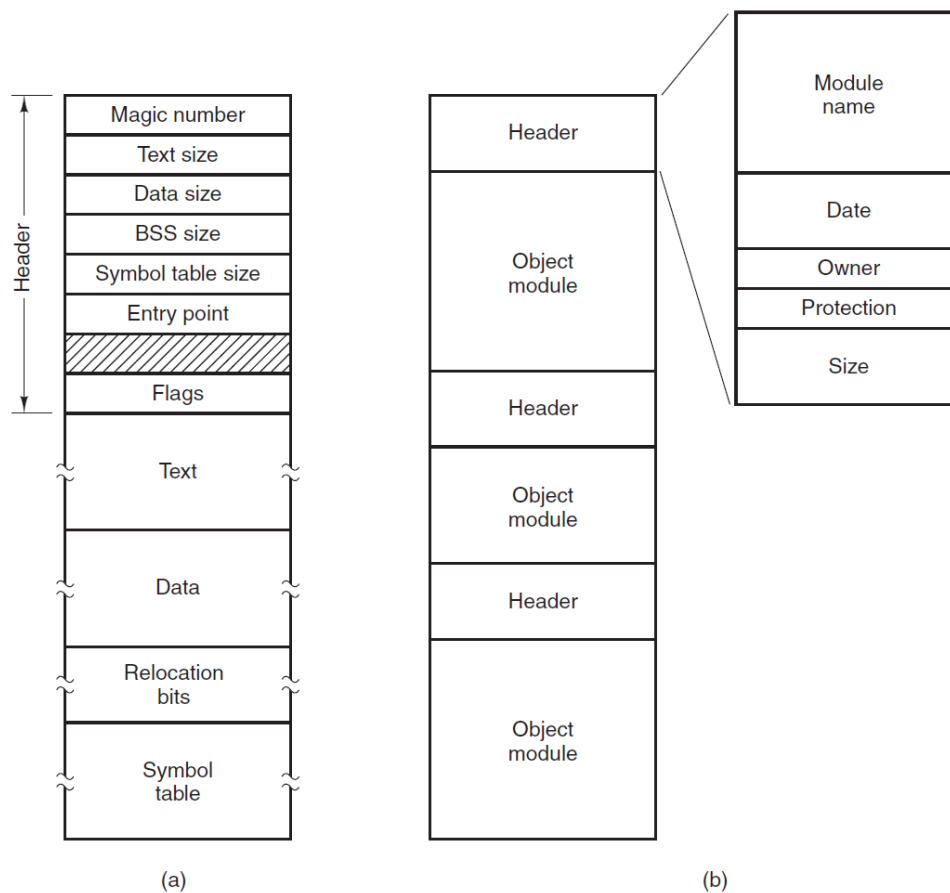


Figure 4-3. (a) An executable file. (b) An archive.

Binary file	Internal structure
Executable file	Contains a header, program text, data, relocation bits, and symbol table.
Archive file	Contains multiple object modules, each preceded by a header.

8 Executable Binary Files

Executable File

An **executable file** is a binary file whose format is recognized by the operating system as runnable program code.

- The OS normally executes a file only if it has the expected executable format.
- The header starts with a **magic number**.
- The magic number identifies the file as an executable and helps prevent accidental execution of non-executable data.
- The header includes sizes of file sections.
- It also includes the program entry point and flags.
- Program text and data are loaded into memory.
- Relocation bits help adjust addresses when loading.
- The symbol table is useful for debugging.

9 Executable File Sections

Section	Purpose
Magic number	Identifies the file as an executable.
Text size	Gives the size of the program instruction section.
Data size	Gives the size of initialized data.
BSS size	Gives the size of uninitialized data.
Symbol table size	Gives the size of debugging/linking symbols.
Entry point	Tells where execution begins.
Flags	Stores executable options or status bits.
Text	Contains program instructions.
Data	Contains program data.
Relocation bits	Used to adjust addresses during loading.
Symbol table	Used mainly for debugging.

10 Archive Binary Files

Archive

An **archive file** is a binary file containing a collection of compiled modules that have not necessarily been linked into one executable.

- UNIX archives can store library procedures or object modules.
- Each module is preceded by a header.
- The header contains metadata about that module.

- This metadata is stored as binary fields.
- Printing the archive as plain text produces unreadable output.

Archive header field	Meaning
Module name	Name of the object module.
Date	Creation or modification date.
Owner	Owner of the module.
Protection	Access protection information.
Size	Size of the module.

11 OS Recognition of File Types

- Every operating system must recognize at least its own executable file type.
- Some operating systems recognize additional file types.
- Recognizing executable format is necessary for safe program loading.
- Some older systems strongly enforced file-type rules.
- TOPS-20 even checked whether source files had changed since the binary was built.
- If the source was newer, it could automatically recompile before running.

12 Strongly Typed Files

Strong Typing

A system has **strongly typed files** when the OS enforces file-type rules, often using mandatory extensions or metadata.

- Strong typing can help beginners avoid mistakes.
- It can also prevent unexpected or creative uses of files.
- Example: a formatter reads a `.c` source file and writes formatted output as `.dat`.
- If the OS refuses to compile `.dat`, the user cannot easily use the formatted output as C source.
- If the OS also refuses to rename `file.dat` to `file.c`, the user is blocked by the type system.

Trade-Off

Strong file typing may protect novice users, but it can frustrate experienced users who need flexibility.

13 ASCII vs. Binary

Feature	ASCII file	Binary file
Contents	Lines of human-readable text.	Arbitrary bytes with program-defined structure.
Display	Can usually be printed or displayed directly.	Usually unreadable if printed directly.
Editing	Can be edited by text editors.	Needs specialized tools or programs.
Interchange	Good for pipelines and manual inspection.	Efficient and structured for specific software.

Final Point

File types help systems and programs decide how to interpret file contents, but too much OS enforcement can reduce flexibility.

File Access

1 Basic Idea

Definition

File access describes the method used to read or write data in a file, especially whether data must be processed in order or can be reached directly.

- Early operating systems mainly supported sequential access.
- Sequential access matched magnetic tape storage.
- Disk storage made random access practical.
- Random access allows programs to jump directly to needed bytes or records.
- Modern systems commonly support both sequential-style reading and explicit repositioning.

2 Sequential Access

Sequential File Access

Sequential access means reading bytes or records in order, starting from the beginning of the file.

- A process reads data in order.
- It cannot skip directly to an arbitrary position.
- It cannot read records out of order.
- The file can usually be rewound.
- After rewinding, the same file can be read again from the beginning.
- This model was convenient for magnetic tape.

Tape Connection

Sequential access fits magnetic tape because tape naturally moves past the read head in order.

3 Random Access

Random-Access File

A **random-access file** is a file whose bytes or records can be read in any order.

- Random access became practical when disks were used for file storage.
- A program can jump to a needed position without reading everything before it.
- Bytes or records can be accessed out of order.
- Some systems also support access by key rather than by numeric position.
- Random access is required by many applications.

4 Why Random Access Matters

Reservation Example

An airline reservation system must access the record for a particular flight directly.

- A customer may ask about one specific flight.
- The reservation program should not read thousands of unrelated flight records first.
- It should jump directly to the needed record.
- This is why database-like applications require random access.

5 Specifying the Start Position

Method	Meaning
Position in every read	Each read operation explicitly gives the file position where reading should begin.
seek operation	A separate operation sets the current file position; later reads continue from that position.

UNIX and Windows

UNIX and Windows use the **seek** model: first set the current position, then read sequentially from there.

6 Seek-Based Access

- The file has a current position.
- A **seek** operation changes that current position.
- After **seek**, the next read starts from the new position.
- Reads can then proceed sequentially from that point.
- This combines random positioning with ordinary sequential reads.

Seek-Then-Read Pattern

```
open(file);  
seek(file, desired_position);  
read(file, buffer, nbytes);
```

7 Sequential vs. Random Access

Feature	Sequential access	Random access
Order	Data are read in order.	Data can be read in any order.
Skipping	Direct skipping is not supported.	Program can jump to a desired location.
Natural medium	Magnetic tape.	Disk and SSD storage.
Typical use	Full file scans, logs, streams.	Databases, indexes, reservation systems.
Current position	Moves forward as data are read.	Can be reset explicitly using position or seek .

8 Key Contrast

- Sequential access is simple and natural for ordered streams.
- Random access is necessary when a program needs one specific record quickly.
- Disks made random access efficient enough for normal file systems.
- The **seek** operation lets a program reposition within a file.
- After seeking, reading can continue sequentially from the new point.

Final Point

Sequential access reads files from beginning to end; random access allows direct movement to the needed byte or record.

File Attributes

1 Basic Idea

Definition

File attributes are extra pieces of information stored by the operating system about a file, in addition to the file name and file data.

- Every file has a name.
- Every file has data.
- Operating systems also store extra information about each file.
- Examples include file size, owner, protection, and modification time.
- These extra items are also called **metadata**.
- The exact set of attributes varies from system to system.

Core Point

Attributes describe and control a file; they are data about the file, not usually the file's user-visible contents.

2 Common File Attributes

Attribute	Meaning
Protection	Who can access the file and in what way.
Password	Password needed to access the file.
Creator	ID of the person who created the file.
Owner	Current owner of the file.
Read-only flag	0 for read/write; 1 for read only.
Hidden flag	0 for normal; 1 for do not display in listings.
System flag	0 for normal file; 1 for system file.
Archive flag	0 for backed up; 1 for needs backup.
ASCII/binary flag	0 for ASCII file; 1 for binary file.
Random access flag	0 for sequential access only; 1 for random access.
Temporary flag	0 for normal; 1 for delete on process exit.
Lock flags	0 for unlocked; nonzero for locked.
Record length	Number of bytes in a record.
Key position	Offset of the key within each record.
Key length	Number of bytes in the key field.
Creation time	Date and time the file was created.
Time of last access	Date and time the file was last accessed.
Time of last change	Date and time the file was last changed.
Current size	Number of bytes currently in the file.
Maximum size	Maximum number of bytes the file may grow to.

3 Protection Attributes

Protection

Protection attributes say who may access a file and what operations they may perform.

- Protection controls access to a file.
- It may specify read, write, execute, append, or delete permissions.
- The creator attribute records who created the file.
- The owner attribute records who currently owns the file.
- Some systems require a password to access a file.
- In those systems, the password is stored as part of the file metadata.

Security Note

Real systems should not store plain-text passwords directly as file attributes. The textbook idea is about access metadata, not a recommended password-storage design.

4 Flag Attributes

Flags

Flags are bits or short fields that enable or describe special file properties.

Flag	Use
Read-only	Prevents writing to the file.
Hidden	Prevents the file from appearing in ordinary file listings.
System	Marks the file as part of the operating system.
Archive	Records whether the file needs backup.
ASCII/binary	Records whether the file is text or binary.
Random access	Records whether random access is supported.
Temporary	Marks the file for deletion when the creating process exits.
Lock	Shows that the file or part of it is locked.

5 Archive Flag

- The archive flag helps backup software.
- A backup program clears the archive flag after backing up a file.
- The operating system sets the archive flag when the file is changed.
- Later, the backup program can check the flag.
- Files with the flag set need to be backed up again.

Backup Use

The archive flag is a small metadata bit that helps backup programs identify changed files.

6 Temporary and Hidden Files

- A hidden file does not appear in normal file listings.
- Hidden files are often used for configuration or system-related data.
- A temporary file may be deleted automatically.
- The temporary flag can tell the OS to delete the file when the creating process terminates.
- These flags affect how the system manages or presents the file.

7 Record and Key Attributes

Record-Key Metadata

Record length, key position, and key length are useful for file systems that support record lookup by key.

- These fields are not needed for ordinary byte-sequence files.
- Record length says how many bytes are in each record.
- Key position says where the key begins inside a record.
- Key length says how many bytes the key field occupies.
- These attributes help the OS find records by key.

8 Time Attributes

Time attribute	Meaning
Creation time	When the file was created.
Time of last access	When the file was most recently read or otherwise accessed.
Time of last change	When the file was most recently modified.

- Time attributes are useful for users, tools, and the OS.
- A compiler can compare source-file time and object-file time.
- If the source file is newer than the object file, recompilation is needed.
- Backup tools can use timestamps to identify changed files.
- File managers can sort files by creation, access, or modification time.

9 Size Attributes

File Size

The current size records how many bytes are currently in the file.

- Current size tells how large the file is now.
- Maximum size tells how large the file is allowed to become.
- Some older mainframe systems required maximum size at file creation time.
- That allowed the OS to reserve storage in advance.
- Modern workstation and personal-computer operating systems usually do not require this.

10 Grouped View

Group	Examples
Protection	Protection, password, creator, owner.
Flags	Read-only, hidden, system, archive, temporary, lock.
Record/key data	Record length, key position, key length.
Times	Creation time, last access time, last change time.
Sizes	Current size, maximum size.

Final Point

File attributes are metadata used to protect, classify, locate, manage, back up, and interpret files.

File Operations

1 Basic Idea

Definition

File operations are system calls or OS services that create, access, modify, inspect, and remove files.

- Files exist to store information.
- File operations allow information to be retrieved later.
- Different operating systems provide different sets of file operations.
- Most systems support a common core set of operations.
- These operations manage file data, file position, attributes, and names.

Core Point

File operations are the interface between programs and persistent file storage.

2 Common File Operations

Operation	Purpose
Create	Make a new empty file and set initial attributes.
Delete	Remove a file and free its disk space.
Open	Prepare a file for use by loading needed metadata into memory.
Close	Finish using a file and release internal OS table space.
Read	Copy data from a file into a caller-provided buffer.
Write	Copy data from a buffer into a file.
Append	Add data only at the end of a file.
Seek	Move the current file position.
Get attributes	Read file metadata such as size or modification time.
Set attributes	Change user-settable metadata such as protection flags.
Rename	Change the name of an existing file.

3 Create and Delete

Create

Create makes a new file with no data.

- The purpose is to announce that a file is coming.

- The OS creates the file-system entry.
- Initial attributes may be set at creation time.
- A newly created file may have size 0.

Delete

Delete removes a file when it is no longer needed.

- Deleting a file frees disk space.
- The file's directory entry or metadata reference is removed.
- Most operating systems provide a system call for deletion.

4 Open and Close

Open

Open prepares a file for later operations by bringing important file information into main memory.

- A process must usually open a file before using it.
- The OS fetches file attributes.
- The OS may fetch disk-address information.
- Keeping this information in memory makes later calls faster.
- The OS may create an entry in an open-file table.

Close

Close ends access to an open file and frees OS resources associated with it.

- Closing frees internal table space.
- Many systems limit how many files one process can keep open.
- Closing may force buffered data to be written to disk.
- Because disks write in blocks, the final partly filled block may be written at close time.

Resource Limit

Forgetting to close files can waste open-file table entries and may hit the process's maximum open-file limit.

5 Read and Write

Read

Read copies data from a file into a buffer supplied by the caller.

- Data usually come from the current file position.
- The caller specifies how many bytes are needed.
- The caller provides a buffer where the data should be placed.
- After the read, the current file position normally advances.

Write

Write copies data from a buffer into a file, usually at the current file position.

- If the current position is at the end of the file, the file grows.
- If the current position is in the middle of the file, existing data are overwritten.
- Overwritten data are lost unless they exist somewhere else.
- After the write, the current file position normally advances.

Overwrite Risk

Writing in the middle of a file replaces existing bytes. This can destroy old data permanently.

6 Append

Append

Append is a restricted form of write that adds data only to the end of a file.

- Append cannot overwrite existing data in the middle of the file.
- It is useful for logs and accumulating output.
- Minimal system-call interfaces may not include a separate append call.
- Some systems provide append anyway because it is convenient and safer for some uses.

7 Seek

Seek

Seek changes the current file position for a random-access file.

- Random-access files need a way to choose where reading or writing begins.
- **seek** moves the file pointer to a specific place in the file.

- After seeking, reads start from the new position.
- Writes also start from the new position.
- UNIX and Windows use this style of file positioning.

Seek Then Read

```
open(file);
seek(file, position);
read(file, buffer, nbytes);
close(file);
```

8 Get and Set Attributes

Attributes

File attributes are metadata such as protection bits, size, owner, and timestamps.

- Programs often need to read file attributes.
- Example: build tools compare modification times.
- The UNIX **make** program checks source-file and object-file timestamps.
- If a source file is newer than its object file, recompilation is needed.
- Some attributes can be changed by users.
- Protection mode and many flags are common user-settable attributes.

Operation	Example use
Get attributes	Read modification time, file size, owner, or protection bits.
Set attributes	Change protection mode, read-only flag, or other user-settable flags.

9 Rename

Rename

Rename changes the name of an existing file.

- Users frequently need to rename files.
- A rename operation changes the file name without rewriting all file data.
- Rename is not always strictly necessary.
- A file can usually be copied to a new file with the new name.
- Then the old file can be deleted.
- Direct rename is more efficient and more convenient.

10 Operation Groups

Group	Operations
Lifecycle	Create, delete, open, close.
Data transfer	Read, write, append.
Positioning	Seek.
Metadata	Get attributes, set attributes.
Naming	Rename.

11 Typical File Use Flow

1. Create or locate the file.
2. Open the file.
3. Read, write, append, or seek as needed.
4. Get or set attributes if needed.
5. Close the file.
6. Delete or rename the file when appropriate.

Final Point

File operations manage the file's lifetime, contents, current position, metadata, and name.

Example Program Using File-System Calls

1 Basic Idea

Goal

The example program copies one UNIX file from a source path to a destination path using ordinary file-system calls.

- The program is called from the command line.
- It opens the source file.
- It creates the destination file.
- It repeatedly reads blocks from the source.
- It writes each block to the destination.
- It closes both files when finished.

Core Point

The program demonstrates how `open`, `creat`, `read`, `write`, and `close` work together in a simple file-copy command.

2 Example Command

```
copyfile abc xyz
```

- `copyfile` is the program name.
- `abc` is the source file.
- `xyz` is the destination file.
- If `xyz` already exists, it is overwritten.
- If `xyz` does not exist, it is created.
- The program must receive exactly two file-name arguments.

3 Full Program

```

/* File copy program. Error checking and reporting is minimal. */
#include <sys/types.h>
#include <fcntl.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[]);

#define BUF_SIZE 4096
#define OUTPUT_MODE 0700

int main(int argc, char *argv[])
{
    int in_fd, out_fd, rd_count, wt_count;
    char buffer[BUF_SIZE];

    if (argc != 3) exit(1);

    in_fd = open(argv[1], O_RDONLY);
    if (in_fd < 0) exit(2);

    out_fd = creat(argv[2], OUTPUT_MODE);
    if (out_fd < 0) exit(3);

    while (1) {
        rd_count = read(in_fd, buffer, BUF_SIZE);
        if (rd_count <= 0) break;

        wt_count = write(out_fd, buffer, rd_count);
        if (wt_count <= 0) exit(4);
    }

    close(in_fd);
    close(out_fd);

    if (rd_count == 0)
        exit(0);
    else
        exit(5);
}

```

4 Header Files and Constants

- The `#include` lines bring in declarations, constants, and function prototypes.
- `<fcntl.h>` provides file-control constants such as `O_RDONLY`.
- `<unistd.h>` provides POSIX system-call interfaces such as `read`, `write`, and `close`.
- `BUF_SIZE` is set to 4096, so the program copies data in 4 KB chunks.
- `OUTPUT_MODE` is set to 0700, which controls the protection bits for the destination file.

Good Practice

Naming constants such as `BUF_SIZE` makes the program easier to read and easier to change later.

5 Command-Line Arguments

argc and argv

`argc` gives the number of command-line strings, and `argv` is an array of pointers to those strings.

For the command:

```
copyfile abc xyz
```

the argument array is:

```
argv[0] = "copyfile";
argv[1] = "abc";
argv[2] = "xyz";
```

- `argc` should be 3.
- `argv[0]` is the program name.
- `argv[1]` is the source file name.
- `argv[2]` is the destination file name.
- If `argc != 3`, the program exits with status code 1.

6 Variables

Variable	Meaning
<code>in_fd</code>	File descriptor for the source file.
<code>out_fd</code>	File descriptor for the destination file.
<code>rd_count</code>	Number of bytes returned by <code>read</code> .
<code>wt_count</code>	Number of bytes returned by <code>write</code> .
<code>buffer</code>	Temporary memory buffer used to hold bytes being copied.

7 Opening and Creating Files

```
in_fd = open(argv[1], O_RDONLY);
if (in_fd < 0) exit(2);

out_fd = creat(argv[2], OUTPUT_MODE);
if (out_fd < 0) exit(3);
```

- `open(argv[1], O_RDONLY)` opens the source file for reading.

- If opening fails, `open` returns a negative value.
- `creat(argv[2], OUTPUT_MODE)` creates or overwrites the destination file.
- If creation fails, `creat` returns a negative value.
- Successful calls return small integers called file descriptors.

File Descriptor

A **file descriptor** is a small integer used by a process to identify an open file in later system calls.

8 Copy Loop

```
while (1) {
    rd_count = read(in_fd, buffer, BUF_SIZE);
    if (rd_count <= 0) break;

    wt_count = write(out_fd, buffer, rd_count);
    if (wt_count <= 0) exit(4);
}
```

- The loop repeatedly tries to read up to 4096 bytes.
- `read` stores the bytes in `buffer`.
- `rd_count` is the number of bytes actually read.
- Usually `rd_count` is 4096.
- On the final successful read, `rd_count` may be less than 4096.
- When end of file is reached, `read` returns 0.
- If `read` returns a negative value, an error occurred.

9 Why write uses rd_count

Important Detail

The program writes `rd_count` bytes, not `BUF_SIZE` bytes, because the last read may return fewer than 4096 bytes.

- Suppose only 1000 bytes remain in the source file.
- `read` returns 1000, not 4096.
- The program must write only those 1000 valid bytes.
- Writing `BUF_SIZE` would copy stale or invalid buffer contents.

10 Closing and Exit Status

```
close(in_fd);
close(out_fd);

if (rd_count == 0)
    exit(0);
else
    exit(5);
```

- Both files are closed after the copy loop ends.
- Closing releases OS open-file table entries.
- Closing may also flush buffered file data.
- If the last read returned 0, the program reached end of file normally.
- If the last read was negative, the program exits with an error code.

11 Exit Codes

Exit code	Meaning
0	Successful copy.
1	Wrong number of command-line arguments.
2	Source file could not be opened.
3	Destination file could not be created.
4	Write error.
5	Read error after the copy loop ended.

12 System Calls Used

Call	Role in the program
open	Opens the existing source file for reading.
creat	Creates or overwrites the destination file.
read	Reads bytes from the source file into memory.
write	Writes bytes from memory into the destination file.
close	Releases the open source and destination files.
exit	Terminates the process with a status code.

13 Program Limitations

- Error checking is minimal.
- Error reporting is poor because it only returns numeric status codes.
- It does not print helpful error messages.
- It assumes **write** writes all requested bytes when it succeeds.

- A production-quality program should handle partial writes.
- A production-quality program should report system error messages.

Final Point

This copy program shows the standard file-copy pattern: open source, create destination, repeatedly read and write blocks, then close both files.

Directory Systems

1 Basic Idea

Definition

A **directory** is a special file used by the file system to keep track of file names and organize files.

- File systems need a way to locate and organize files.
- Directories, also called folders, provide this organization.
- A directory may contain file names.
- In modern systems, a directory may also contain other directories.
- Directory organization affects how easily users can find and group files.

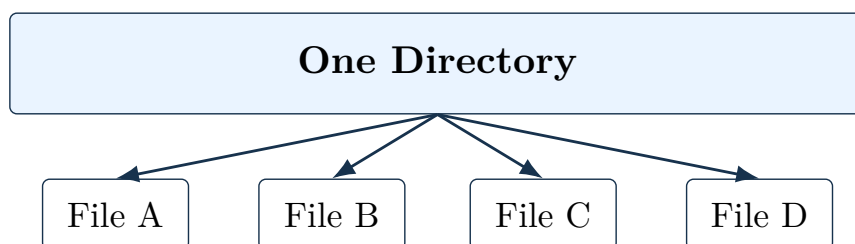
Core Point

Directories are themselves files, but their contents are interpreted by the file system as file-system organization data.

2 Single-Level Directory System

Single-Level Directory

A **single-level directory system** has one directory containing all files in the system.



- There is only one directory.
- Every file is listed in that directory.
- Since there is only one directory, the term root directory is not very important.
- Early personal computers often used this design.
- The CDC 6600 supercomputer also used a single directory, even with many users.
- The main reason was software simplicity.

3 Advantages of Single-Level Directories

- Very simple to implement.
- Very simple to understand.
- File lookup is straightforward.
- There is only one place to search.
- Still useful for very small systems.
- Sometimes used in embedded devices, digital cameras, or simple music players.

4 Limitations of Single-Level Directories

Main Problem

A single directory becomes hard to use when there are many files or many users.

- All files share one namespace.
- File-name conflicts are more likely.
- Related files cannot be grouped naturally.
- Finding a file becomes difficult when thousands of files exist.
- It does not match how modern users organize work.

5 Need for Hierarchy

Motivation

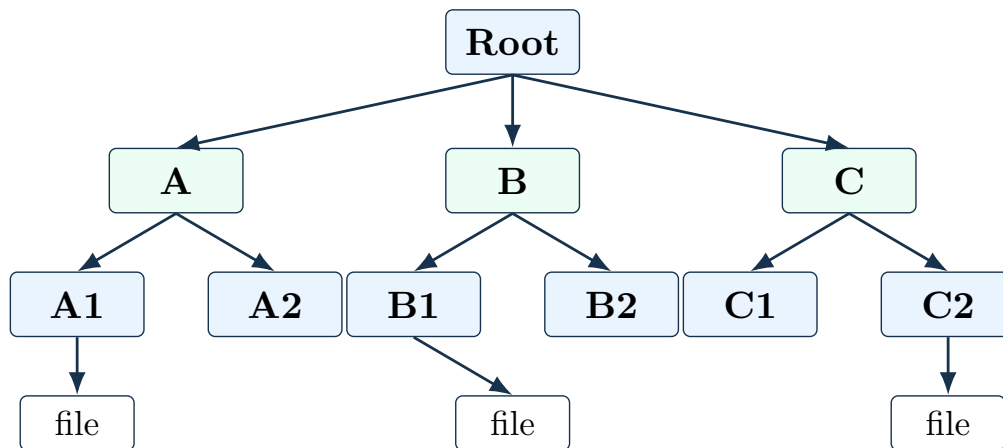
Users need to group related files together, especially when they have many projects or many categories of work.

- A user may have files for a book.
- The same user may also have student programs.
- Other files may belong to compiler projects, grant proposals, email, meeting minutes, papers, or games.
- Keeping all of these in one directory would be confusing.
- A hierarchy lets each group have its own directory.

6 Hierarchical Directory System

Hierarchical Directory

A **hierarchical directory system** organizes directories as a tree, with directories containing files and subdirectories.



- The directory structure forms a tree.
- The root directory is at the top.
- Directories below the root can belong to different users.
- Each user can create private subdirectories.
- Subdirectories can represent projects, courses, documents, or any other grouping.
- Files are placed inside the relevant directory.

7 Benefits of Hierarchical Directories

- Users can create as many directories as needed.
- Related files can be grouped together.
- File-name conflicts are reduced because names are local to directories.
- Multiple users can share one file server while keeping separate directory trees.
- Large collections of files become easier to navigate.
- This structure is used by nearly all modern file systems.

Modern Design

Hierarchical directories are standard in modern systems because they scale from a few files to millions of files across many users.

8 Single-Level vs. Hierarchical

Feature	Single-level directory	Hierarchical directory
Structure	One directory contains every file.	Directories form a tree.
Simplicity	Very simple.	More complex but more useful.
Grouping	No natural grouping.	Related files can be grouped in subdirectories.
Naming conflicts	More likely.	Less likely because directories provide separate contexts.
Best use	Very small or embedded systems.	Modern multiuser and general-purpose systems.

9 Key Contrast

- A single-level directory is easy to implement but does not scale well.
- A hierarchical directory is more complex but much better for organization.
- Early systems used single directories because storage and software were simpler.
- Modern systems use directory trees because users manage many files and many projects.

Final Point

Directories organize file names. Single-level directories are simple, but hierarchical directories provide the structure needed for modern file systems.

Path Names

1 Basic Idea

Definition

A **path name** is a name that identifies a file or directory by describing its location in a directory tree.

- Hierarchical file systems organize files as a tree of directories.
- A path name tells the operating system how to reach a file in that tree.
- Two common forms are absolute path names and relative path names.
- Absolute paths start from the root directory.
- Relative paths start from the current working directory.

2 Absolute Path Names

Absolute Path

An **absolute path name** gives the complete path from the root directory to the target file or directory.

- Absolute paths always start at the root directory.
- They are unique within a file-system tree.
- They do not depend on the process's current working directory.
- Example: `/usr/ast/mailbox`.
- This means: `root` $\rightarrow$ `usr` $\rightarrow$ `ast` $\rightarrow$ `mailbox`.

Rule

If the first character of a path is the path separator, the path is absolute.

3 Path Separators

System	Example absolute path
UNIX	<code>/usr/ast/mailbox</code>
Windows	<code>\usr\ast\mailbox</code>
MULTICS	<code>>usr>ast>mailbox</code>

- UNIX separates path components with `/`.

- Windows separates path components with backslash.
- MULTICS used >.
- The separator character differs, but the idea is the same.

4 Relative Path Names

Relative Path

A **relative path name** identifies a file relative to the process's current working directory.

- A relative path does not begin at the root directory.
- It is interpreted starting from the current working directory.
- If the current working directory is `/usr/ast`, then `mailbox` refers to `/usr/ast/mailbox`.
- Relative paths are often shorter and more convenient.
- Their meaning can change when the working directory changes.

5 Working Directory

Working Directory

The **working directory**, also called the current directory, is the directory used as the starting point for relative path names.

- Each process has its own working directory.
- Changing one process's working directory does not affect other processes.
- When the process exits, its working-directory change leaves no trace in the file system.
- A program can change its working directory when convenient.
- Library procedures should be careful: changing the working directory can surprise the rest of the program.

Library Warning

If a library procedure changes the working directory, it should change it back before returning.

6 Absolute vs. Relative Example

If the working directory is `/usr/ast`, these commands do the same thing:

```
cp /usr/ast/mailbox /usr/ast/mailbox.bak
cp mailbox mailbox.bak
```

- The first command uses absolute paths.
- The second command uses relative paths.
- Both copy `/usr/ast/mailbox` to `/usr/ast/mailbox.bak`.

7 When to Use Absolute Paths

- Use an absolute path when the program must access a specific file regardless of the working directory.
- Example: a spelling checker may need `/usr/lib/dictionary`.
- The spelling checker may be started from any directory.
- Therefore, using the full absolute path is safer.
- Absolute paths are less convenient to type but more predictable.

8 Changing Directory Instead

- A program that needs many files from one directory can change its working directory.
- Example: it can change to `/usr/lib`.
- Then it can open `dictionary` using a relative name.
- This works because the program explicitly knows where it is in the directory tree.

9 Dot and Dotdot

Special Directory Entries

Most hierarchical file systems provide `.` and `..` in every directory.

Entry	Meaning
<code>.</code>	The current directory.
<code>..</code>	The parent directory. In the root directory, it usually refers to the root itself.

- Dot is pronounced “dot”.
- Dotdot is pronounced “dot dot”.
- `..` lets a path move upward in the directory tree.
- `.` is useful when a command expects a directory name.

10 UNIX Directory Tree Example

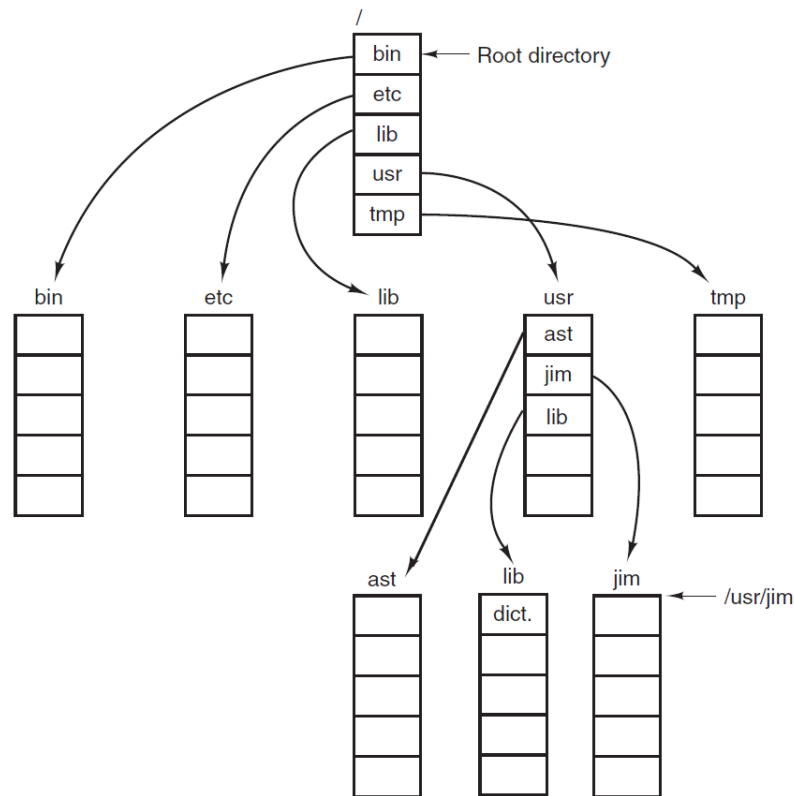


Figure 4-8. A UNIX directory tree.

Assume the current working directory is `/usr/ast`. The file `/usr/lib/dictionary` can be copied into the current directory using:

```
cp ../lib/dictionary .
```

- `..` moves from `/usr/ast` up to `/usr`.
- `lib` moves down into `/usr/lib`.
- `dictionary` selects the source file.
- `.` names the current directory, `/usr/ast`, as the destination.

11 Equivalent Copy Commands

These commands have the same effect when the working directory is `/usr/ast`:

```
cp ../lib/dictionary .
cp /usr/lib/dictionary .
cp /usr/lib/dictionary dictionary
cp /usr/lib/dictionary /usr/ast/dictionary
```

- The first command uses `..` and `..`.

- The second uses an absolute source and current-directory destination.
- The third gives the destination file name explicitly.
- The fourth gives both source and destination as absolute paths.
- All identify the same source and destination.

12 Comparison

Feature	Absolute path	Relative path
Starting point	Root directory.	Current working directory.
Uniqueness	Unique independent of process location.	Meaning depends on working directory.
Convenience	Longer to type.	Usually shorter to type.
Best use	Programs needing a fixed file.	Interactive work inside a known directory.

Final Point

Absolute paths are complete and independent of the working directory; relative paths are shorter but depend on where the process currently is.

Directory Operations

1 Basic Idea

Definition

Directory operations are system calls used to create, remove, open, read, rename, and link directory entries.

- Directory system calls vary more across operating systems than ordinary file calls.
- UNIX provides a useful model for understanding common directory operations.
- Directories are special files, but they should not be treated like ordinary byte streams by user programs.
- Directory operations manage names and links between names and files.

2 Common Directory Operations

Operation	Meaning
Create	Create a new empty directory.
Delete	Remove an empty directory.
Opendir	Open a directory so its entries can be read.
Closedir	Close an open directory and free internal table space.
Readdir	Return the next entry from an open directory.
Rename	Change the name of a directory.
Link	Add another directory entry for an existing file.
Unlink	Remove a directory entry.

3 Create

Create Directory

Creating a directory makes a new directory that is empty except for `.` and `...`

- The new directory initially contains no user files.
- The special entry `.` names the directory itself.
- The special entry `..` names the parent directory.
- These entries are usually inserted automatically by the system.
- In a few systems, the `mkdir` program may create them.

4 Delete

Delete Directory

Deleting a directory removes it from the file system, but normally only if it is empty.

- A directory can be deleted only when it is empty.
- A directory containing only `.` and `..` is considered empty.
- The entries `.` and `..` cannot be deleted as ordinary entries.
- Requiring emptiness prevents accidental deletion of a whole subtree.

Safety Rule

Deleting a nonempty directory is normally rejected because it would otherwise remove names for files and subdirectories below it.

5 Opendir and Closedir

Reading Directories

A directory must be opened before a program can read its entries.

- A listing program opens a directory to read the names it contains.
- Opening a directory is analogous to opening a file before reading it.
- After the directory has been read, it should be closed.
- Closing frees internal operating-system table space.

Call	Purpose
<code>opendir</code>	Prepare a directory for entry-by-entry reading.
<code>closedir</code>	Finish directory reading and release OS resources.

6 Readdir

Readdir

`readdir` returns the next entry in an open directory using a standard format.

- Directory entries can be read one at a time.
- Older systems sometimes allowed ordinary `read` on directories.
- That forced programmers to understand the internal directory format.

- `readdir` hides the internal directory representation.
- It returns directory entries in a standard form.
- This works even if file systems use different internal directory structures.

Main Benefit

`readdir` gives programs a portable way to read directory entries without depending on raw directory layout.

7 Rename

Rename Directory

Directories can often be renamed much like ordinary files.

- Rename changes a directory's name.
- It does not require copying all files inside the directory.
- The directory entry is updated.
- The actual directory contents remain in place.

8 Link

Hard Link

A **hard link** creates another directory entry that refers to the same existing file.

- Linking lets the same file appear in more than one directory.
- The system call specifies an existing file and a new path name.
- The new path becomes another name for the same file.
- The file's i-node link count is incremented.
- The link count tracks how many directory entries refer to the file.
- This kind of link is called a hard link.

9 Unlink

Unlink

Unlink removes a directory entry.

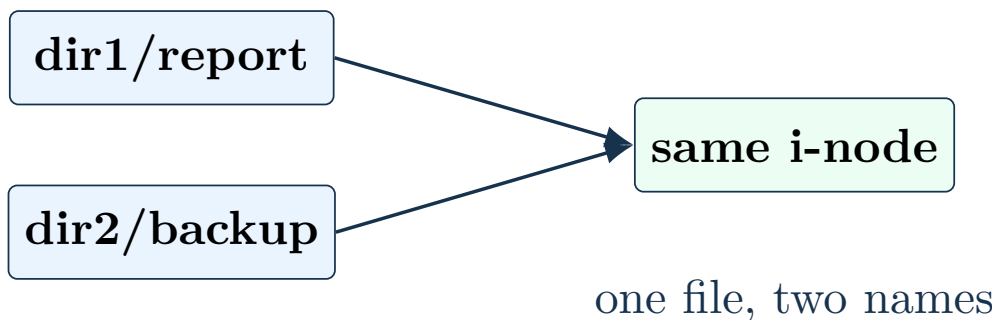
- If the file has only one directory entry, unlinking removes it from the file system.

- If the file has multiple hard links, only the specified path name is removed.
- Other path names for the same file remain valid.
- In UNIX, the system call used to delete ordinary files is actually **unlink**.

Link Count Rule

A file's storage can be freed only after its last directory entry has been unlinked.

10 Hard Link Example



Linux Command Example: Hard Link

The command `ln original hardlink` creates a second name for the same file object.

```

$ mkdir link-demo
$ cd link-demo
$ echo "Operating Systems" > notes.txt

$ ln notes.txt notes-hard.txt
$ ls -li
12501 -rw-r--r-- 2 ast ast 18 Jul 31 10:00 notes-hard.txt
12501 -rw-r--r-- 2 ast ast 18 Jul 31 10:00 notes.txt
  
```

- The first number is the i-node number.
- Both names show the same i-node number: 12501.
- The link count is 2, because two directory entries point to the same file.

```

$ echo "File Systems" >> notes-hard.txt
$ cat notes.txt
Operating Systems
File Systems

$ rm notes.txt
$ cat notes-hard.txt
Operating Systems
File Systems
  
```

What This Shows

Editing through either hard-link name changes the same underlying file. Removing one name does not delete the file while another hard link still exists.

Hard Link Limits

A hard link normally cannot point to a directory and cannot cross file-system boundaries. It works by adding another directory entry to the same i-node.

11 Symbolic Links

Symbolic Link

A **symbolic link** is a special file that contains the path name of another file.

- A symbolic link does not point directly to the same internal file structure.
- Instead, it names another file by path.
- When the symbolic link is opened, the file system reads the stored path.
- Then lookup starts again using that path.
- Symbolic links can cross disk boundaries.
- They can also refer to files on remote computers.
- They are usually less efficient than hard links.

Linux Command Example: Symbolic Link

The command `ln -s target linkname` creates a symbolic link whose contents are the target path.

```
$ echo "directory chapter" > chapter.txt
$ ln -s chapter.txt chapter-link.txt
$ ls -li
12502 -rw-r--r-- 1 ast ast 18 Jul 31 10:05 chapter.txt
12503 lrwxrwxrwx 1 ast ast 11 Jul 31 10:05 chapter-link.txt -> chapter.txt
```

- The symbolic link has its own i-node: 12503.
- The target file has a different i-node: 12502.
- The `l` at the start of `lrwxrwxrwx` means the entry is a symbolic link.
- The arrow `chapter-link.txt -> chapter.txt` shows the stored path.

```
$ cat chapter-link.txt
directory chapter

$ echo "updated through symlink" >> chapter-link.txt
$ cat chapter.txt
directory chapter
updated through symlink
```

What This Shows

Opening a symlink usually follows the stored path and operates on the target file.

```
$ rm chapter.txt
$ cat chapter-link.txt
cat: chapter-link.txt: No such file or directory

$ ls -l chapter-link.txt
chapter-link.txt -> chapter.txt
```

Dangling Symlink

After the target path is removed, the symbolic link still exists, but it points to a name that no longer resolves. This is called a dangling symbolic link.

12 Hard Links vs. Symbolic Links

Feature	Hard link	Symbolic link
Target	Same internal file object or i-node.	A path name stored in a small special file.
Lookup	No second path lookup is needed.	File system follows the stored path and restarts lookup.
Disk boundary	Usually cannot cross file-system boundaries.	Can cross disk and network boundaries.
Efficiency	More efficient.	Less efficient because path lookup is repeated.
Failure case	Remains valid while the underlying file still has links.	Can become dangling if the target path disappears.

13 Operation Groups

Group	Operations
Lifecycle	Create, delete.
Traversal	Opendir, readdir, closedir.
Naming	Rename.
Multiple names	Link, unlink.
Protection	Other system-specific calls may manage directory permissions.

Final Point

Directory operations manage names, directory contents, and links. Hard links add another name for the same file; symbolic links store a path to another file.

File-System Layout

1 Basic Idea

File-System Layout

A file system is stored inside a disk partition. The partition contains special control blocks for booting, describing the file system, tracking free space, describing files, and storing ordinary directories and file data.

- A physical disk can be divided into one or more **partitions**.
- Each partition can contain an independent file system.
- The whole disk contains boot information, while each partition contains its own internal file-system layout.
- The exact layout differs between file systems, but the same basic ideas appear repeatedly.

2 Disk-Level Layout

Master Boot Record

Sector 0 of the disk is called the **MBR**, or **Master Boot Record**. It is used during system startup.

- The **MBR** contains a small boot program.
- The end of the MBR contains the **partition table**.
- The partition table records the starting and ending addresses of each partition.
- One partition is marked as **active**.
- The active partition is the partition from which the operating system is loaded.

3 Boot Sequence

1. The computer starts and the BIOS runs.
2. The BIOS reads sector 0, which contains the MBR.
3. The BIOS executes the program stored in the MBR.
4. The MBR program finds the active partition from the partition table.
5. The MBR reads the first block of the active partition.
6. This first block is the **boot block**.
7. The boot block program loads the operating system stored in that partition.

Important Detail

Every partition normally begins with a boot block, even if that partition does not currently contain a bootable operating system. This keeps partition layout uniform and allows the partition to become bootable later.

4 Typical Partition Layout

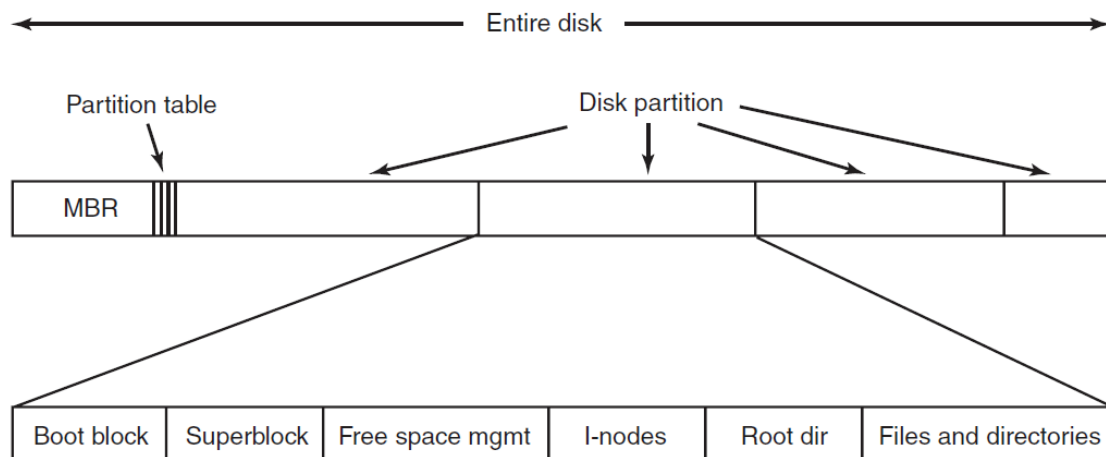


Figure 4-9. A possible file-system layout.

Figure 1: A possible file-system layout inside a disk partition.

Component	Purpose
Boot block	Contains code used to start an operating system from the partition.
Superblock	Stores key parameters that describe the file system.
Free-space management	Tracks which disk blocks are free and available for allocation.
I-nodes	Store per-file information, such as attributes and block locations.
Root directory	Stores the top-level directory of the file-system tree.
Files and directories	Store the remaining ordinary file data and subdirectories.

5 Superblock

Superblock

The **superblock** contains the main administrative information needed to understand and manage the file system.

- It is read into memory when the system boots or when the file system is first accessed.
- It usually contains a **magic number** identifying the file-system type.
- It records the number of blocks in the file system.

- It also stores other key parameters needed by the operating system.

6 Free-Space Management

- The file system must know which blocks are free before it can create or extend files.
- Free space may be represented using a **bitmap**.
- It may also be represented using a **list of pointers** to free blocks.
- Without free-space information, the system cannot safely allocate new disk blocks.

7 I-Nodes and Root Directory

- The **i-node area** is an array of data structures.
- Each i-node describes one file.
- An i-node stores information about a file rather than the file's name.
- The **root directory** is the starting point of the directory tree.
- From the root directory, the operating system can locate other directories and files.

8 Key Points

- The MBR belongs to the whole disk.
- The partition table describes where partitions begin and end.
- A boot block belongs to a partition.
- A superblock describes one file system.
- Free-space management tells the system where new data can be placed.
- I-nodes describe files.
- The root directory is the entry point into the directory hierarchy.

Final Point

A file-system layout separates boot information, file-system metadata, free-space tracking, file descriptions, and actual file data so that the operating system can boot, mount, allocate, locate, and manage files efficiently.

Contiguous Allocation

1 Basic Idea

Contiguous Allocation

In contiguous allocation, each file is stored as one continuous run of disk blocks. A file's disk location can be described using only the starting block and the number of blocks in the file.

- The disk is divided into fixed-size blocks.
- A file occupies consecutive blocks, with no gaps between its blocks.
- Example: with 1-KB blocks, a 50-KB file needs 50 consecutive blocks.
- Example: with 2-KB blocks, a 50-KB file needs 25 consecutive blocks.
- If the file does not exactly fill its last block, the unused part of that block is wasted.

2 Figure Example

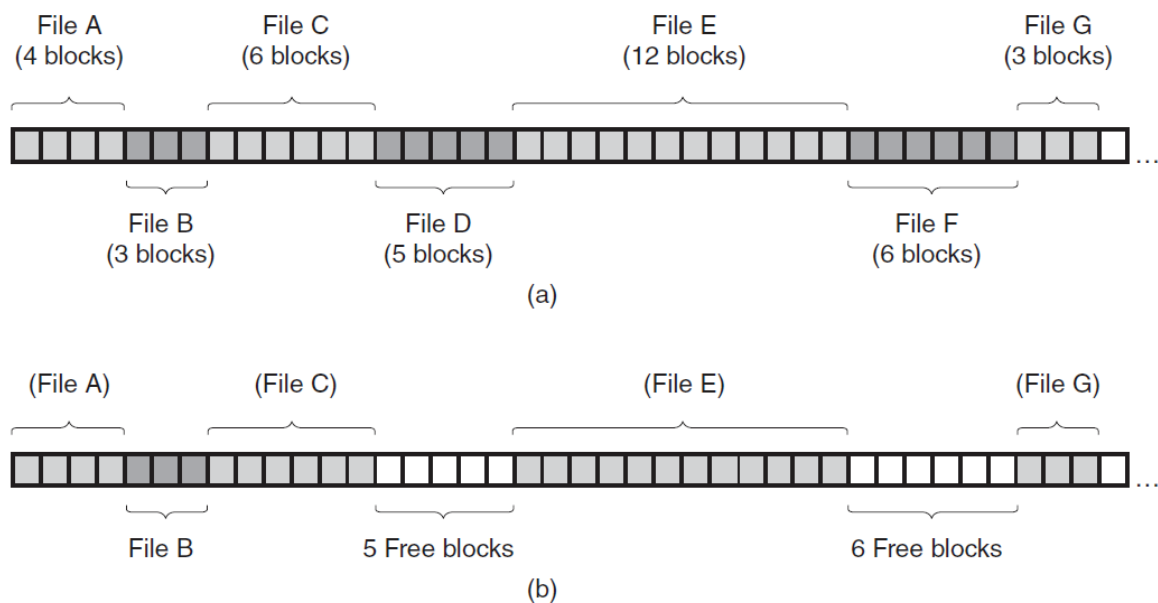


Figure 4-10. (a) Contiguous allocation of disk space for seven files. (b) The state of the disk after files *D* and *F* have been removed.

Figure 1: Contiguous allocation before and after file deletion.

- In part (a), files are stored one after another in consecutive disk blocks.
- File A uses 4 blocks.

- File B uses 3 blocks.
- File C uses 6 blocks.
- File D uses 5 blocks.
- File E uses 12 blocks.
- File F uses 6 blocks.
- File G uses 3 blocks.
- In part (b), files D and F have been removed.
- Their deleted regions become holes of 5 free blocks and 6 free blocks.

3 How the System Tracks a File

Metadata Needed

For each file, the file system only needs:

starting block number + file length in blocks

- If the first block is block b , then the next block is $b + 1$.
- The i th block of the file is found by simple addition.
- This makes the allocation method very easy to implement.

4 Advantages

Advantage	Explanation
Simple metadata	Only the first block and file length must be stored.
Fast lookup	Any file block can be found by addition.
High read speed	The whole file can be read sequentially from disk.
Few seeks	Only one seek is needed to reach the first block.
Good bandwidth	After the first block is reached, data can be read at disk bandwidth.

5 Main Problem: Fragmentation

External Fragmentation

Over time, contiguous allocation creates holes between files. These holes may be free, but they may not be large enough for a new file that needs one long continuous run of blocks.

- When a file is deleted, its blocks become free.
- The disk is not compacted immediately because moving many later blocks is too expensive.

- After many create and delete operations, the disk contains a mixture of files and holes.
- New files must fit into a hole large enough to hold the whole file contiguously.
- This creates external fragmentation.

6 Why File Size Is a Problem

- To choose a suitable hole, the system needs to know the final size of the file.
- This is unrealistic for many files because users and programs often do not know the final size in advance.
- If the estimate is too small, the file may run out of space.
- If the estimate is too large, the system may fail to find a large enough hole.
- Compacting the disk to remove holes can require copying huge amounts of data.

7 Where Contiguous Allocation Works Well

Best Use Case

Contiguous allocation works well when file sizes are known in advance and files do not change after being written.

- It is suitable for CD-ROMs because all file sizes are known before the disk is created.
- It is also useful for write-once optical media such as DVDs and Blu-ray disks.
- DVD movie data may be stored as several contiguous pieces called **extents**.
- An extent is one contiguous physical part of a larger logical file.

8 Comparison

Point	Contiguous Allocation
Storage rule	Store each file in consecutive disk blocks.
Metadata	Starting block and file length.
Performance	Excellent for sequential reads.
Main cost	External fragmentation.
Hard question	How large will the file become?
Good fit	Read-only or write-once media.
Poor fit	General-purpose disks with growing and deleting files.

Final Point

Contiguous allocation is simple and fast, but it is hard to use for changing file systems because deleted files create holes and new files require one large enough continuous free region.

Linked-List Allocation

1 Basic Idea

Linked-List Allocation

In linked-list allocation, a file is stored as a chain of disk blocks. Each block contains a pointer to the next block of the same file, so the file blocks do not have to be contiguous on disk.

- A file is divided into disk blocks.
- The blocks may be located anywhere on the disk.
- The first word of each block stores a pointer to the next block.
- The remaining part of the block stores file data.
- The last block contains a special end marker instead of a pointer to another block.
- The directory entry only needs to store the address of the first block.

2 Linked List of Disk Blocks

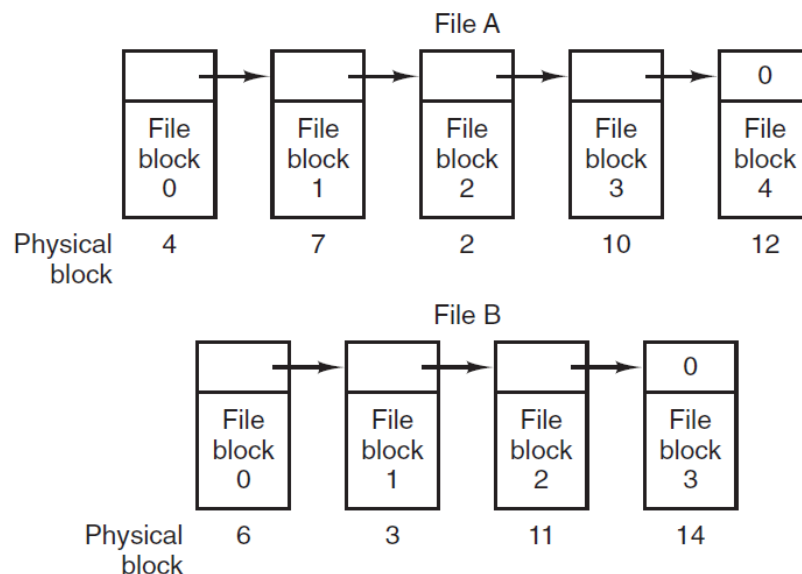


Figure 4-11. Storing a file as a linked list of disk blocks.

Figure 1: Storing files as linked lists of disk blocks.

- File A uses physical blocks $4 \rightarrow 7 \rightarrow 2 \rightarrow 10 \rightarrow 12$.
- File B uses physical blocks $6 \rightarrow 3 \rightarrow 11 \rightarrow 14$.
- The physical blocks of a file do not need to be next to each other.
- The chain is followed from the first block until the end marker is found.

3 Advantages

Advantage	Explanation
No external fragmentation	Any free disk block can be used for any file.
Simple directory entry	The directory only stores the first block number.
Files can grow easily	A new block can be linked onto the end of the file.
Good sequential access	Reading from beginning to end follows the block chain naturally.

4 Disadvantages

Slow Random Access

Random access is very slow because the system must follow the chain from the beginning to find a later block.

- To read block n , the operating system must first read blocks 1 through $n - 1$.
- This makes direct access to the middle of a file expensive.
- Each block loses a few bytes because the pointer is stored inside the block.
- The usable data area is no longer a clean power-of-two size.
- If programs expect full block-sized reads and writes, the system may need extra copying and concatenation.

5 Linked-List Allocation Using a Table

File Allocation Table

A **File Allocation Table**, or **FAT**, stores the next-block pointers in a memory table instead of storing them inside the disk blocks.

- The FAT has one entry for each physical disk block.
- Each entry tells which block comes next in the file.
- A special value such as -1 marks the end of a file.
- Unused blocks have no active chain entry.
- The directory still needs only the starting block of the file.

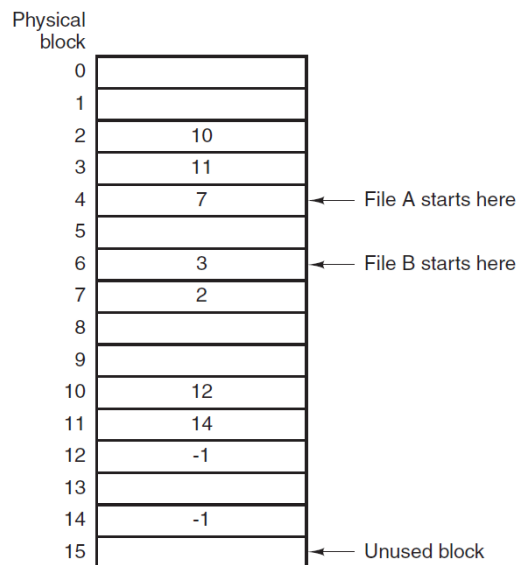


Figure 4-12. Linked-list allocation using a file-allocation table in main memory.

Figure 2: Linked-list allocation using a file-allocation table in memory.

6 FAT Example

File	Chain from FAT
File A	4 → 7 → 2 → 10 → 12 → -1
File B	6 → 3 → 11 → 14 → -1

- To locate File A, the directory gives starting block 4.
- FAT entry 4 points to 7.
- FAT entry 7 points to 2.
- The chain continues until entry 12 gives -1.
- The same idea locates File B starting from block 6.

7 Why FAT Improves the Basic Method

FAT Improvement

FAT removes the pointer from the disk block, so the entire block can store data. It also makes chain traversal faster because the chain can be followed in memory.

- Random access still requires following the chain.
- However, following the chain in memory is much faster than reading each pointer from disk.
- Full disk blocks remain available for file data.
- The file system still needs only the first block number in the directory entry.

8 Main FAT Limitation

Memory Cost

The whole FAT must be kept in memory for efficient operation. This becomes impractical on very large disks.

- A 1-TB disk with 1-KB blocks has about 1 billion blocks.
- The FAT therefore needs about 1 billion table entries.
- If each entry uses 3 bytes, the table uses about 3 GB of memory.
- If each entry uses 4 bytes for faster lookup, it uses about 4 GB of memory.
- This memory requirement shows why FAT does not scale well to large disks.

9 Comparison

Point	Basic Linked List	FAT-Based Linked List
Pointer location	First word of each disk block	Separate table in memory
Data space per block	Reduced by pointer size	Whole block available for data
Sequential access	Good	Good
Random access	Very slow, requires disk reads along the chain	Faster, chain is followed in memory
Directory entry	Stores first block	Stores first block
Main weakness	Poor random access and awkward block size	Large memory requirement

Final Point

Linked-list allocation avoids external fragmentation and allows files to grow easily, but basic linked lists make random access slow. FAT improves access by moving pointers into memory, but the table can become too large for modern large disks.

I-Nodes

1 Basic Idea

I-Node

An **i-node**, or **index-node**, is a per-file data structure that stores a file's attributes and the disk addresses of the blocks belonging to that file.

- Each file has its own i-node.
- The i-node tells the operating system where the file's blocks are located on disk.
- Given the i-node, the file system can find all blocks of the file.
- The file name is usually stored in a directory entry, while the file's metadata and block addresses are stored in the i-node.

2 I-Node Structure

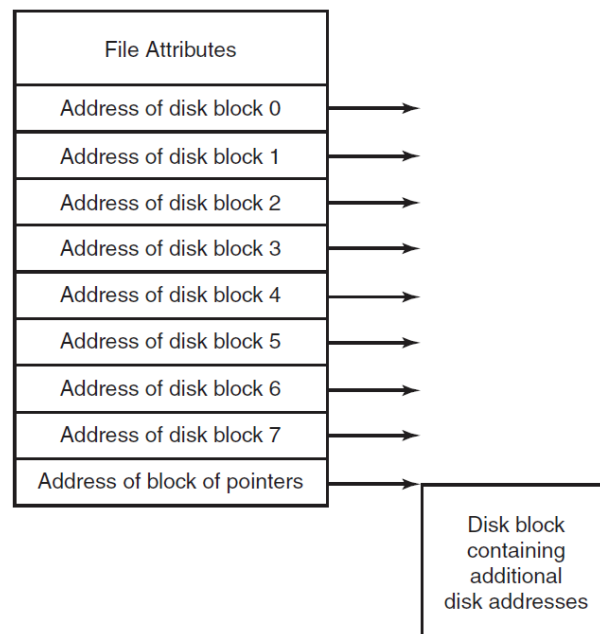


Figure 4-13. An example i-node.

Figure 1: An example i-node with direct block addresses and one pointer block.

- The top part stores **file attributes**.
- Attributes may include file size, owner, permissions, and time information.
- The next fields store direct addresses of disk blocks.

- Each direct address points to one data block of the file.
- The final address may point to a block that contains more disk-block addresses.

3 How Blocks Are Found

Lookup Rule

To access a file, the operating system first finds the file's i-node. The i-node then gives the disk block addresses needed to read or write the file data.

1. A directory entry maps the file name to its i-node.
2. The file system loads the i-node.
3. The i-node's direct block addresses point to the first data blocks.
4. If the file is larger, an indirect pointer points to a block containing additional addresses.
5. The operating system follows these addresses to reach the required file data.

4 Main Advantage

Memory Efficiency

An i-node only needs to be in memory while its file is open. This makes i-nodes more scalable than a full in-memory file allocation table.

- A FAT-style table has one entry for every disk block.
- Therefore, FAT memory use grows with disk size.
- With i-nodes, memory use grows mainly with the number of open files.
- If each i-node has size n bytes and at most k files can be open, the needed memory is:

Open I-Node Memory

$$\text{memory for open i-nodes} = k \times n \text{ bytes}$$

- This amount is usually much smaller than storing a full table for every disk block.
- The disk may be 100 GB, 1000 GB, or 10,000 GB, but the open i-node table depends on open files, not total disk blocks.

5 Problem: Very Large Files

Fixed Address Limit

If an i-node has room for only a fixed number of disk addresses, then a file can grow beyond the direct addresses stored in the i-node.

- A small file can be represented using only direct block addresses.
- A larger file needs more block addresses than the i-node can store directly.
- One solution is to use the last i-node address as an **indirect pointer**.
- The indirect pointer points to a disk block full of additional disk-block addresses.
- More advanced systems may use multiple levels of indirection.

6 Direct and Indirect Addresses

Address Type	Meaning
Direct address	Points directly to a file data block.
Indirect address	Points to a block that contains more disk-block addresses.
Double indirect address	Points to a block whose entries point to more address blocks.
Triple indirect address	Adds another level for very large files.

7 Comparison with FAT

Point	FAT	I-Nodes
Main structure	One table entry per disk block	One structure per file
Memory use	Proportional to disk size	Proportional to open files
When loaded	Normally kept in memory	Loaded when file is open
Scaling	Poor for very large disks	Better for large disks
File growth	Follow table chain	Use direct and indirect addresses

8 Key Points

- An i-node stores file attributes and block addresses.
- It separates file naming from file storage information.
- It only has to be in memory when the file is open.
- It scales better than keeping one memory-table entry for every disk block.
- Indirect blocks allow the scheme to support large files.

Final Point

I-nodes solve the scaling problem of a full in-memory allocation table by keeping per-file block information in an i-node and loading only the i-nodes for currently open files.

Implementing Directories

1 Basic Idea

Directory Implementation

A directory maps a human-readable file name to the information needed to locate that file's data on disk.

- Before a file can be read, it must be opened.
- During open, the operating system uses the path name to find the file's directory entry.
- The directory entry provides the information needed to locate the file's disk blocks.
- The exact location information depends on the allocation method used by the file system.

2 What a Directory Entry Stores

Allocation Method	Directory Entry May Store
Contiguous allocation	Disk address of the first block and file length.
Linked-list allocation	Disk address of the first block.
FAT-based allocation	Starting block number; the rest is found from the FAT.
I-node allocation	I-node number.

Main Function

The directory system maps the file's ASCII name to the internal information required to find the file data.

3 Where File Attributes Are Stored

Attributes Inside Directory Entries

- A directory can be stored as a list of fixed-size entries.
- Each entry contains a fixed-length file name.
- Each entry also contains file attributes.
- The entry may contain one or more disk addresses.
- This design is simple, but directory entries can become large.

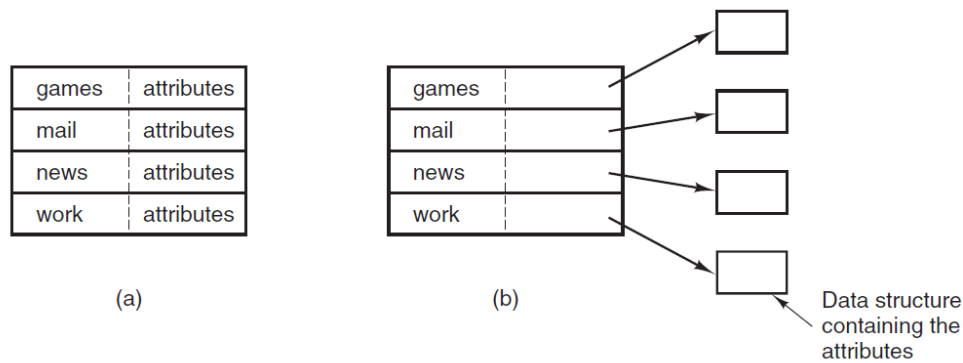


Figure 4-14. (a) A simple directory containing fixed-size entries with the disk addresses and attributes in the directory entry. (b) A directory in which each entry just refers to an i-node.

Figure 1: Two common ways to store directory entries and file attributes.

Attributes Inside I-Nodes

- In i-node-based systems, attributes are stored in the i-node.
- The directory entry only stores the file name and i-node number.
- This keeps directory entries shorter.
- It also separates naming from file metadata and block-location information.

4 Fixed-Length File Names

- Older systems often used short fixed-length names.
- MS-DOS used a 1–8 character base name plus an optional 1–3 character extension.
- UNIX Version 7 allowed names of 1–14 characters.
- A simple modern approach is to reserve a large fixed field, such as 255 characters, for every file name.
- This is easy to implement but wastes directory space because most names are much shorter than the maximum.

5 Variable-Length File Names

Method 1: In-Line Names

- Directory entries are no longer all the same size.
- Each entry begins with a fixed part, such as entry length and file attributes.
- The actual file name is stored directly after the fixed part.
- The name is usually terminated by a special character such as 0.
- Padding may be added so the next entry begins on a word boundary.

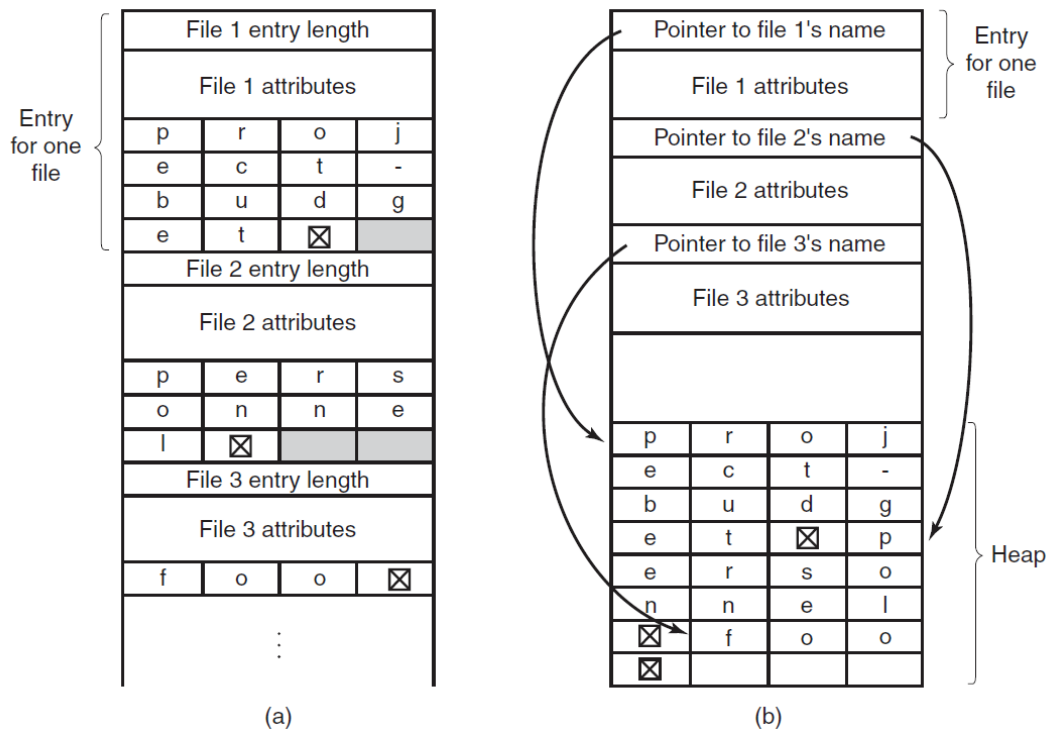


Figure 4-15. Two ways of handling long file names in a directory. (a) In-line. (b) In a heap.

Figure 2: Two ways of storing long variable-length file names in directories.

In-Line Name Problem

When a file is deleted, it leaves a variable-sized gap in the directory. A later file name may not fit into that gap without compacting the directory.

Method 2: Names in a Heap

- Directory entries remain fixed length.
- Each entry stores a pointer to the file name.
- The actual file names are stored together in a heap at the end of the directory.
- When a directory entry is removed, another fixed-size entry can reuse its place.
- No padding is required after file names because names do not need to start on word boundaries.

6 Comparing Long-Name Designs

Point	In-Line Names	Heap Names
Entry size	Variable length	Fixed length
Name location	Inside the directory entry	In a heap at the end
Deletion effect	Leaves variable-sized gaps	Fixed entries are easier to reuse
Padding	Often needed for word alignment	Usually not needed
Extra management	Directory compaction may be needed	Heap management is needed
Page-fault risk	Possible while reading long names	Also possible while reading heap names

7 Searching Directories

- The simplest directory lookup method is linear search.
- The operating system checks entries from the beginning until it finds the requested name.
- Linear search is acceptable for small directories.
- It becomes slow when a directory contains hundreds or thousands of files.

8 Hash Tables for Fast Lookup

Directory Hash Table

A directory can use a hash table to map file names quickly to directory entries.

- The file name is hashed to a value between 0 and $n - 1$, where n is the hash-table size.
- The hash value selects a slot in the table.
- If the slot is empty, the directory entry pointer can be stored there.
- If multiple names hash to the same slot, a linked list of entries is used.
- Lookup hashes the name and searches only the chain for that hash slot.

Hash Table Tradeoff

Hash tables make directory lookup faster, but they make directory administration more complex.

9 Caching Directory Lookups

- Another way to speed up lookup is to cache recent search results.
- Before scanning a directory, the file system first checks whether the name is already in the cache.

- If the name is cached, the file can be located immediately.
- Caching works best when a small set of files accounts for most lookups.

10 Key Points

- Directories translate file names into file-location information.
- Attributes may be stored directly in the directory entry or separately in an i-node.
- Fixed-length file names are simple but waste space.
- Variable-length file names save space but complicate directory management.
- Hashing and caching improve lookup performance for large or frequently used directories.

Final Point

Implementing directories is mainly about choosing how to store names, attributes, and location information so that file lookup is correct, compact, and fast enough for real workloads.

Shared Files

1 Basic Idea

Shared File

A **shared file** is a file that appears in more than one directory, so different users or directories can access the same underlying file.

- Shared files are useful when multiple users work on the same project.
- A shared file may appear in directories belonging to different users.
- The connection from another directory to the shared file is called a **link**.
- With links, the file-system structure is no longer a pure tree.
- It becomes a **directed acyclic graph**, or **DAG**.

2 Shared File as a DAG

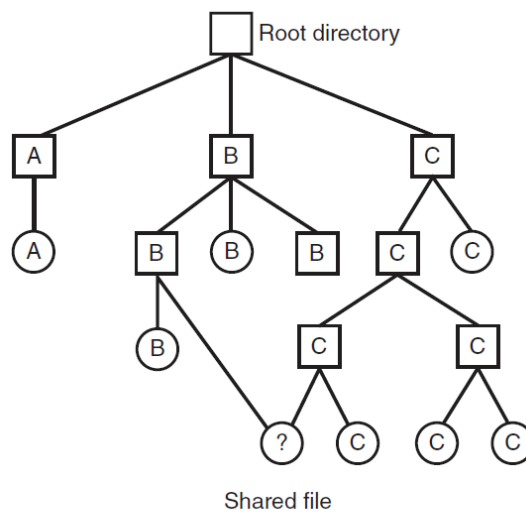


Figure 4-16. File system containing a shared file.

Figure 1: A file system containing a shared file.

- In a tree, each file or directory has exactly one parent path.
- With sharing, the same file can be reached through more than one directory.
- The shared file has multiple incoming links.
- This makes maintenance more complicated because the system must handle multiple names for the same file.

3 Why Direct Disk Addresses Cause Trouble

Problem with Copying Addresses

If directory entries contain direct disk addresses, linking a file by copying those addresses can break sharing.

- Suppose C owns a file and B links to it.
- If disk addresses are copied into B's directory, B and C now have separate address lists.
- If B appends data, only B's directory entry may be updated.
- If C appends data, only C's directory entry may be updated.
- The two users may stop seeing the same file contents, defeating the purpose of sharing.

4 Solution 1: Hard Links

Hard Link

A **hard link** makes multiple directory entries point to the same internal file data structure, such as an i-node.

- Disk block addresses are stored in the file's data structure, not directly in each directory entry.
- In UNIX-like systems, this data structure is the i-node.
- Each directory entry points to the same i-node.
- Since the i-node is shared, updates to the file are visible through all hard links.
- The i-node stores a **link count**.
- The link count records how many directory entries point to the file.

5 Hard-Link Count Example

- Before linking, only C's directory points to the file, so the count is 1.
- After B creates a hard link, both B's and C's directories point to the same i-node, so the count becomes 2.
- If C removes the file name from C's directory, the i-node is not deleted.
- The count decreases to 1 because B's directory still points to the file.
- The file is deleted only when the link count becomes 0.

6 Hard-Link Drawback

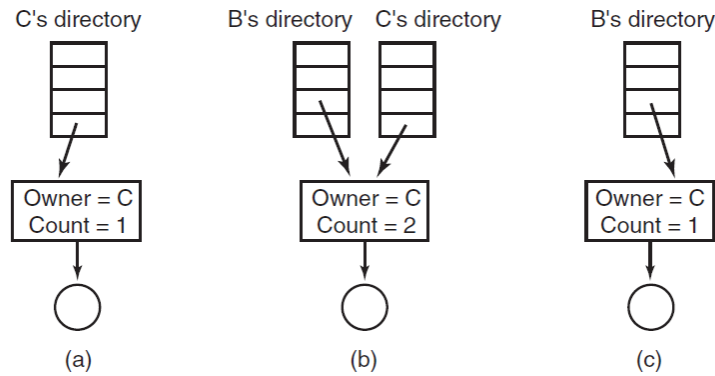


Figure 4-17. (a) Situation prior to linking. (b) After the link is created. (c) After the original owner removes the file.

Figure 2: Link count before linking, after linking, and after the original owner removes the file.

Ownership and Accounting Problem

With hard links, the file may continue to be charged to the original owner even after that owner's directory entry has been removed.

- Creating a hard link does not change the owner recorded in the i-node.
- If C owns the file and B links to it, the owner remains C.
- If C deletes C's name for the file, B may still keep the file alive.
- The system cannot easily find and remove every directory entry that points to the i-node.
- This can affect accounting and disk quotas.

7 Solution 2: Symbolic Links

Symbolic Link

A **symbolic link** is a small special file that stores the path name of another file.

- B can link to C's file by creating a new file of type **LINK**.
- The link file stores the path name of the target file.
- When the link is opened, the operating system reads the path and follows it to the real file.
- The symbolic link does not point directly to the target file's i-node.
- Removing the symbolic link does not affect the target file.

8 Symbolic-Link Behavior

- If the original file is deleted, the symbolic link remains.

- Later attempts to open the symbolic link fail because the target path no longer exists.
- This is called a dangling or broken symbolic link.
- Symbolic links can cross disk boundaries.
- They can also refer to files on remote machines if the path includes network location information.

9 Hard Links vs Symbolic Links

Point	Hard Link	Symbolic Link
What it stores	Direct reference to the same i-node	Path name of the target file
Target deletion	File remains until link count is 0	Link becomes broken if target is deleted
Effect of removing link	Decreases link count	Removes only the link file
Disk boundaries	Usually limited within one file system	Can cross disks and networks
Overhead	Low lookup overhead	Extra lookup and path parsing
Extra storage	No separate path file needed	Needs a link object and stored path

10 General Problems with Links

- A linked file can have more than one path name.
- Programs that recursively scan directories may find the same file multiple times.
- Backup programs may accidentally store multiple copies of one shared file.
- Restoring such backups on another system may duplicate the file instead of preserving the link.
- File-system tools must be aware of links to avoid incorrect accounting, copying, or traversal.

Final Point

Shared files make collaboration easier by allowing multiple directory entries to refer to the same file, but links complicate deletion, ownership, accounting, traversal, and backup behavior.

Log-Structured File Systems

1 Basic Idea

Log-Structured File System

A **log-structured file system**, or **LFS**, treats the disk as one large log. Instead of writing small updates in many different disk locations, it collects pending writes in memory and writes them to disk in large contiguous segments.

- LFS was designed to improve file-system performance when disks have slow seeks.
- The main idea came from Berkeley research by Rosenblum and Ousterhout.
- It is especially focused on making small random writes more efficient.
- The disk is written sequentially, like appending records to a log.

2 Why LFS Was Needed

- CPUs keep getting faster.
- Main memory keeps getting larger.
- Disk capacity keeps growing.
- Disk seek time does not improve as quickly.
- Larger memory allows larger file-system caches.
- Many read requests can be served from cache without disk access.
- Therefore, many real disk accesses become writes rather than reads.

Small-Write Problem

Small disk writes are inefficient because a tiny write may still require a seek and rotational delay before the actual data transfer.

3 Cost of Small Writes

- A small disk write may take only about 50 microseconds of actual transfer time.
- It may be preceded by about 10 milliseconds of seek time.
- It may also wait about 4 milliseconds for rotational delay.
- The disk spends most of the time positioning, not transferring data.
- This can reduce disk efficiency to a fraction of 1 percent.

4 Why Ordinary File Systems Do Many Small Writes

Example: Creating a UNIX File

Creating a new file can require several separate writes, even before much user data is written.

- The directory i-node may need to be updated.
- The directory block must record the new file name.
- The new file's i-node must be written.
- The file's data block must be written.
- These writes affect different disk locations.
- Delaying them can create consistency problems if the system crashes.
- For this reason, important metadata writes are often done quickly.

5 Main LFS Mechanism

Write Strategy

LFS buffers many small writes in memory, collects them into a large segment, and writes that segment sequentially to the end of the log.

1. File-system updates are first buffered in memory.
2. Periodically, the buffered updates are grouped into one segment.
3. The segment is written contiguously at the end of the log.
4. The segment may contain file data, directory blocks, i-nodes, and other metadata.
5. A segment summary is placed at the start of the segment.
6. The segment summary describes what is stored inside the segment.

6 Segments

- A segment is a large contiguous unit written to disk.
- A segment may contain different kinds of blocks mixed together.
- Common segment contents include:
 - data blocks,
 - directory blocks,
 - i-nodes,
 - segment summary information.
- If segments are large enough, such as about 1 MB, the disk can be used close to full bandwidth.

7 I-Nodes in LFS

- LFS still uses i-nodes.
- The i-node structure can be similar to UNIX i-nodes.
- The difference is that i-nodes are not stored at fixed disk positions.
- They are scattered throughout the log.
- Once an i-node is found, its file-block addresses are used normally.

8 I-Node Map

I-Node Map

An **i-node map** is an index that maps an i-node number to the current disk location of that i-node.

- In traditional UNIX, an i-node's disk address can be calculated from its i-node number.
- In LFS, i-nodes move because new versions are appended to the log.
- Therefore, LFS needs an i-node map.
- Entry i in the map points to i-node i on disk.
- The map is stored on disk.
- Frequently used parts of the map are cached in memory.

9 Opening a File in LFS

1. The path name is searched to find the file's i-node number.
2. The i-node map is consulted.
3. The map gives the current disk address of the i-node.
4. The i-node is read.
5. The i-node gives the disk addresses of the file's data blocks.
6. The required file blocks are then read from their locations in the log.

10 Why Cleaning Is Needed

Finite Disk Problem

If the disk were infinite, LFS could keep appending forever. Real disks are finite, so old and obsolete log segments must eventually be reclaimed.

- When a file block is overwritten, the new version is appended to the log.
- The old version still occupies space in an older segment.
- The current i-node points to the new block, not the old block.
- The old block is now obsolete garbage.
- LFS needs a way to reclaim such space.

11 Cleaner Thread

Cleaner

The **cleaner** is a background thread that scans old log segments, keeps live data, discards obsolete data, and marks old segments free.

1. The cleaner scans the log circularly.
2. It reads the segment summary of an old segment.
3. It checks which i-nodes and blocks are described there.
4. It consults the current i-node map to see whether those items are still live.
5. Obsolete blocks are discarded.
6. Live blocks and i-nodes are copied into memory.
7. The old segment is marked free.
8. The live information is later written into a new segment at the front of the log.

12 Disk as a Circular Buffer

Circular Log View

LFS treats the disk like a circular buffer: the writer adds new segments at the front, while the cleaner frees old segments from the back.

- The writer thread appends new segments.
- The cleaner thread reclaims old segments.
- Live data from old segments is rewritten into newer segments.
- The i-node map must be updated when i-nodes move.
- This bookkeeping is complex but enables high write throughput.

13 Advantages and Costs

Point	Advantage	Cost
Write performance	Converts many small writes into large sequential writes	Requires buffering and segment management
Disk bandwidth	Can use nearly full disk bandwidth for segments	Depends on producing large enough segments
Reads	Many reads can be served from cache	Uncached reads may find blocks scattered in the log
Metadata	I-nodes can keep familiar structure	I-node locations require an i-node map
Space reuse	Cleaner reclaims obsolete blocks	Cleaning adds background work and bookkeeping

14 Key Points

- LFS is designed for systems where disk seeks are the bottleneck.
- It assumes many reads can be satisfied from cache.
- It improves small-write performance by writing large contiguous segments.
- It stores i-nodes and data blocks throughout the log.
- It uses an i-node map to locate the current copy of each i-node.
- It uses a cleaner to reclaim obsolete log segments.

Final Point

Log-structured file systems trade implementation complexity for faster write performance by turning many small random writes into large sequential log writes.

Journaling File Systems

1 Basic Idea

Journaling File System

A **journaling file system** writes a log record of planned file-system changes before applying those changes to the main file system. If the system crashes, the log tells the operating system what operation was in progress.

- Journaling is used to improve file-system reliability after crashes.
- The file system first records what it is about to do.
- Only after the log entry is safely written does it modify the real file-system structures.
- After the operation finishes successfully, the log entry can be erased or marked complete.
- During reboot, the system checks the journal and completes any unfinished operations.

2 Why Journaling Is Needed

Crash Consistency Problem

Many file-system operations require several separate updates. If the system crashes after some updates but before others, the file system can become inconsistent.

- A disk update is not usually a single all-or-nothing action.
- File metadata and file data may be stored in different disk blocks.
- Directory entries, i-nodes, free-block maps, and free-i-node lists may all need updates.
- A crash in the middle can leave some structures updated and others old.
- Journaling gives the recovery code a clear record of what must be completed.

3 Example: Removing a File

UNIX File Removal

Removing a file in a UNIX-style file system requires several metadata updates.

1. Remove the file's entry from its directory.
2. Release the file's i-node to the pool of free i-nodes.
3. Return the file's disk blocks to the pool of free disk blocks.

- If all three steps complete, the file is removed correctly.
- If a crash happens between steps, the order matters.
- Different crash points can create different kinds of corruption or resource loss.

4 Crash Cases Without Journaling

Crash Point	Resulting Problem
After removing directory entry only	The i-node and blocks are unreachable but not free, so disk resources are lost.
After removing directory entry and freeing i-node	The data blocks may remain lost.
After freeing i-node first	The old directory entry may point to an i-node later reused by another file.
After freeing blocks first	A valid i-node may point to blocks that are already in the free pool.

Worst Case

If blocks are returned to the free pool before the i-node is cleared, two or more files may later point to the same disk blocks.

5 How Journaling Handles the Operation

Journal First

The key rule is: write the plan to the journal before changing the actual file-system structures.

1. Write a journal entry describing all required actions.
2. Force the journal entry to disk.
3. Optionally read it back to verify that it was written correctly.
4. Perform the real file-system updates.
5. After all updates finish, erase or mark the journal entry complete.

6 Recovery After a Crash

- During reboot, the file system checks the journal.
- If no incomplete operation is recorded, normal mounting can continue.
- If a pending operation is found, the file system reruns the logged actions.
- The actions may be repeated after repeated crashes.
- When the operation is finally complete, the journal entry is cleared.

7 Idempotent Operations

Idempotent Operation

An operation is **idempotent** if it can be repeated many times and still produces the same final state without causing harm.

- Journaling depends on logged operations being idempotent.
- Recovery may repeat an operation that was already partly completed.
- Repeating the operation must not damage the file system.
- File-system data structures must be designed so journal actions are safe to replay.

Operation	Idempotent?
Mark i-node k as free in a bitmap	Yes, setting the same bit repeatedly is harmless.
Mark block n as free in a bitmap	Yes, the final bitmap state is unchanged by repetition.
Remove any directory entry named foobar	Yes, once it is gone, removing it again has no extra effect.
Append block n to the end of a free list	No, it may add the same block more than once.
Add block n only if it is not already on the free list	Yes, but it is more expensive.

8 Atomic Transactions

Atomic Transaction

An **atomic transaction** groups several file-system actions so the system completes either all of them or none of them.

- A transaction begins with a begin-transaction operation.
- It ends with an end-transaction operation.
- The file system treats all operations inside the transaction as one logical unit.
- After a crash, recovery must not leave only part of the transaction applied.
- Atomic transactions make crash recovery more systematic.

9 Examples of Journaling File Systems

- NTFS uses an extensive journaling system.
- Linux ext3 added journaling while remaining compatible with ext2.
- ReiserFS was an early Linux journaling file system.
- OS X provides journaling as an option for supported file systems.

10 Journaling vs Log-Structured File Systems

Point	Journaling File System	Log-Structured File System
Main purpose	Crash recovery and consistency	High write throughput
Log role	Records planned metadata changes	Stores the main file-system contents
Compatibility	Easier to add to existing designs	Often requires a different layout
Recovery	Replay or finish logged operations	Use log and cleaner metadata
Common use	Widely used in practical file systems	Less widely used directly

11 Key Points

- Journaling protects the file system from inconsistent partial updates.
- The journal records the intended operation before the real update happens.
- Crash recovery checks the journal and completes unfinished work.
- Logged operations must be idempotent so they can be safely repeated.
- Atomic transactions group related updates into all-or-nothing units.

Final Point

Journaling file systems make crash recovery fast and reliable by recording file-system intentions before applying them and replaying safe, idempotent operations after a failure.

Virtual File Systems

1 Basic Idea

Virtual File System

A **Virtual File System**, or **VFS**, is an operating-system layer that provides one common interface for many different concrete file systems.

- A computer may use several file systems at the same time.
- Examples include NTFS, FAT, ext2, ext3, ReiserFS, ISO 9660, and network file systems.
- The VFS hides many differences between these concrete file systems.
- User programs call standard file operations such as `open`, `read`, `write`, and `lseek`.
- The VFS decides which underlying file system should handle each request.

2 Windows and UNIX Approaches

Point	Windows Style	UNIX Style
User view	Different file systems often appear as different drive letters, such as C: and D:.	Multiple file systems are integrated into one directory hierarchy.
Mounting idea	The drive letter identifies the target file system.	File systems are mounted at directories such as /usr, /home, or /mnt.
Visibility	The distinction is more visible to users.	Users see one unified file-system tree.
Example	NTFS on C:, FAT on D:, DVD as another drive.	ext2 root, ext3 mounted on /usr, ReiserFS on /home, CD-ROM on /mnt.

3 Position of the VFS

- User processes make standard POSIX calls.
- The POSIX interface forms the upper interface of the VFS.
- The VFS layer receives the request first.
- The VFS forwards the request to the correct concrete file system.
- The concrete file systems use their own internal code and buffer cache to perform the real work.

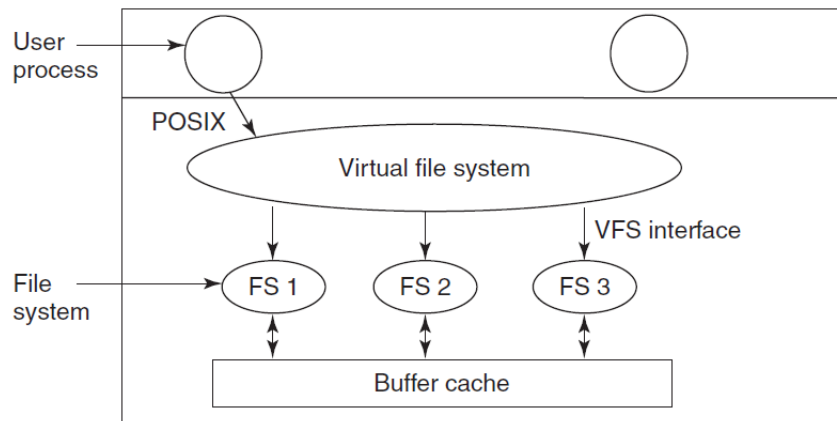


Figure 4-18. Position of the virtual file system.

Figure 1: Position of the virtual file system between POSIX calls and concrete file systems.

4 Two VFS Interfaces

VFS Interfaces

The VFS has an upper interface for user processes and a lower interface for concrete file systems.

Interface	Purpose
Upper interface	Provides standard system calls such as <code>open</code> , <code>read</code> , <code>write</code> , <code>close</code> , and <code>lseek</code> .
Lower interface	Defines functions that concrete file systems must provide to the VFS.

- The lower interface is sometimes called the **VFS interface**.
- A new file system must implement the functions expected by the VFS.
- These functions may include operations for reading blocks, writing blocks, opening files, reading directories, and managing metadata.
- The VFS does not need to know whether the data is stored on a local disk, USB drive, CD-ROM, or remote server.

5 Important VFS Objects

- VFS implementations are often object-oriented in design, even when written in C.
- A **superblock** describes a mounted file system.
- A **v-node** describes an open or referenced file.
- A **directory object** describes a file-system directory.
- Each object has associated operations supplied by the concrete file system.
- The VFS also maintains internal structures such as the mount table and file-descriptor tables.

6 Registration and Mounting

File-System Registration

When a concrete file system is mounted, it registers with the VFS by providing function pointers for the operations the VFS expects.

1. At boot time, the root file system registers with the VFS.
2. Later, other file systems may be mounted.
3. Each mounted file system provides a table of function addresses.
4. The VFS stores these addresses.
5. When the VFS needs an operation, it calls the correct function through the table.

7 Opening a File Through VFS

Example

Suppose a file system is mounted on `/usr`, and a process calls:

```
open("/usr/include/unistd.h", O_RDONLY)
```

1. The VFS begins parsing the path.
2. It notices that another file system is mounted on `/usr`.
3. It finds that mounted file system's superblock.
4. It starts lookup inside the root directory of the mounted file system.
5. It resolves `include/unistd.h`.
6. It creates a v-node in memory.
7. It asks the concrete file system for the information from the file's i-node.
8. It stores the needed information in the v-node.
9. It creates an entry in the calling process's file-descriptor table.
10. It returns a file descriptor to the process.

8 Reading a File Through VFS

1. The process calls `read` using a file descriptor.
2. The VFS uses the process table and file-descriptor table to find the open-file state.
3. The open-file state points to the v-node.
4. The v-node points to a table of function pointers.
5. The VFS calls the concrete file system's read function.
6. The concrete file system locates and returns the requested data.

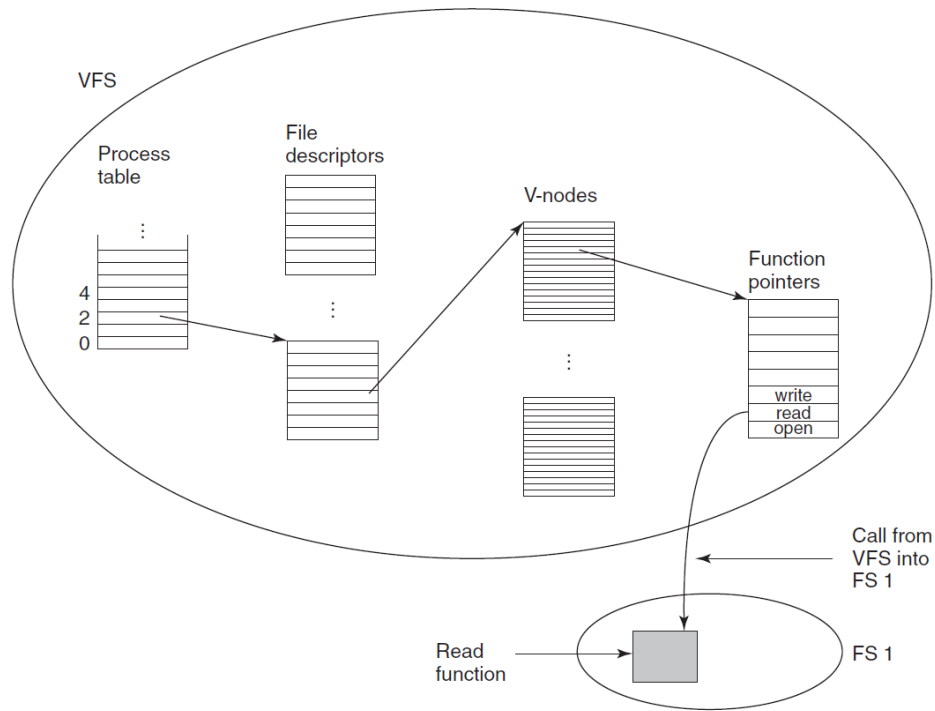


Figure 4-19. A simplified view of the data structures and code used by the VFS and concrete file system to do a read.

Figure 2: Simplified data structures and code path used by the VFS and concrete file system during a read.

9 Why VFS Is Useful

Benefit	Explanation
Uniform API	Programs use the same system calls for different file systems.
Mount integration	Different file systems can appear inside one directory tree.
Extensibility	New file systems can be added by implementing the VFS interface.
Location transparency	The VFS does not care whether data is local, removable, optical, or remote.
Cleaner kernel design	Common file-system behavior is centralized in the VFS layer.

10 Key Points

- VFS abstracts the common part of file-system behavior.
- It receives standard POSIX file calls from user processes.
- It forwards operations to the correct concrete file system.
- Concrete file systems register function tables with the VFS.
- V-nodes represent files at the VFS level.
- Superblocks represent mounted file systems.

- The design makes it easier to support many file systems in one operating system.

Final Point

The VFS layer lets one operating system present a unified file interface while delegating real storage work to many different concrete file systems.

Disk-Space Management

1 Basic Idea

Definition

Disk-space management decides how file-system blocks are allocated, freed, tracked, and limited per user.

Main Trade-Off

Small blocks save disk space but reduce performance. Large blocks improve sequential I/O but waste space in small files.

2 File Storage Strategies

Strategy	Advantage	Problem
Contiguous allocation	Fast sequential access.	File growth may require moving the file.
Fixed-size blocks	Handles growth and avoids large moves.	File may be scattered over disk.

Memory Analogy

Contiguous files resemble segmentation; fixed-size disk blocks resemble paging.

3 Block Size Choice

Block size	Good for	Bad for
Small	Space utilization; less waste in last block.	Large-file performance; more seeks and metadata.
Large	Sequential throughput; fewer block accesses.	Small files; more internal fragmentation.

Typical Result

Historically, file systems often used 1 KB to 4 KB blocks. Larger disks make larger blocks more acceptable, but workloads still matter.

4 File-Size Distribution

File size	VU 1984	VU 2005	Web
1 KB or less	48.05	30.91	47.82
4 KB or less	75.31	59.13	70.64
64 KB or less	99.18	90.84	94.80
1 MB or less	100.00	98.87	99.62

Key Observation

Many files are small, but most disk blocks may still be used by a small number of large files.

5 Disk Access Formula

Given Disk

Assume 1 MB per track, rotation time 8.33 msec, and average seek time 5 msec.

Time to Read k Bytes

$$T(k) = 5 + 4.165 + \left(\frac{k}{1,000,000} \right) 8.33 \text{ msec}$$

- 5 msec = average seek time.
- 4.165 msec = half a rotation on average.
- Transfer time is usually tiny for small blocks.

Example

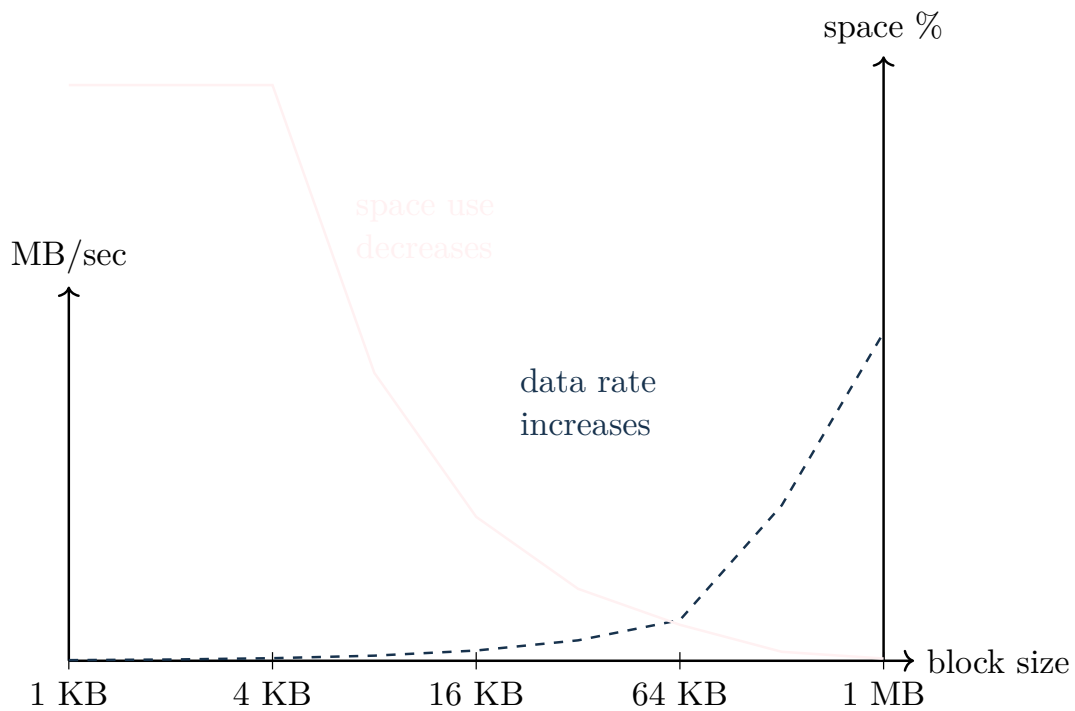
For a 4 KB block, $T \approx 9.20$ msec and the data rate is only about 0.45 MB/sec because seek and rotation dominate.

6 Space Efficiency

$$\text{Efficiency} = \frac{\text{useful file size}}{\text{allocated space}} \times 100\%$$

Block size	Allocated for 4 KB file	Efficiency
1 KB	4 KB	100%
4 KB	4 KB	100%
8 KB	8 KB	50%
16 KB	16 KB	25%
64 KB	64 KB	6.25%

7 Performance vs. Space



8 Tracking Free Blocks

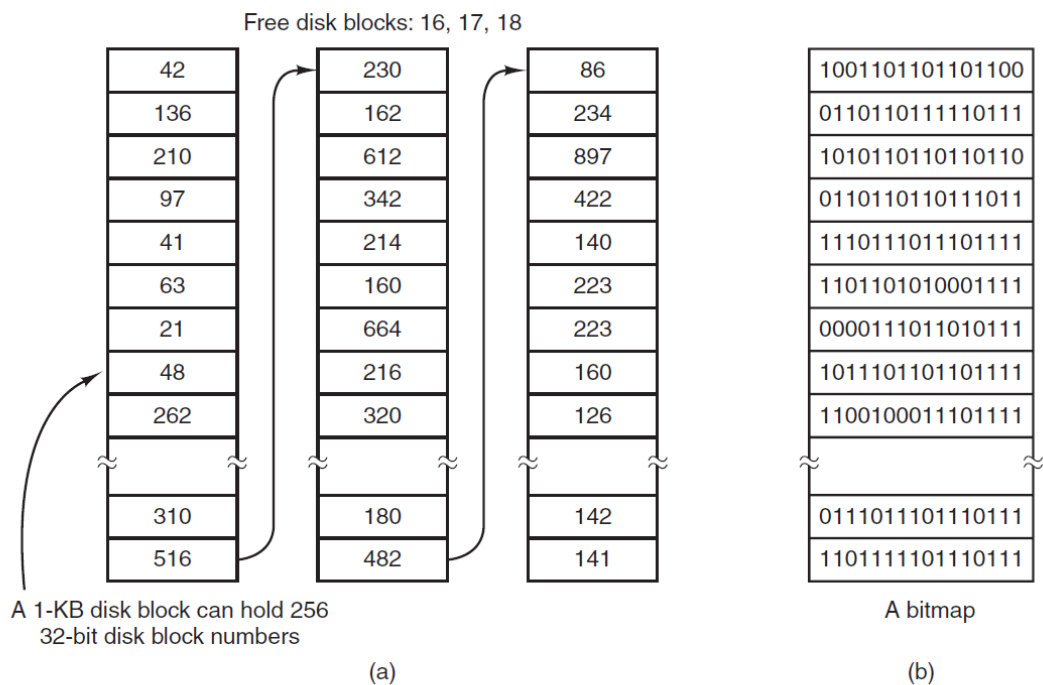


Figure 4-22. (a) Storing the free list on a linked list. (b) A bitmap.

Method	Main idea	Note
Free list	Free blocks store addresses of other free blocks.	Simple; only good when many free blocks need not be searched by position.
Bitmap	One bit per disk block.	Compact; good for finding nearby free blocks.
Runs	Store start block plus count.	Excellent for long free runs; poor if disk is fragmented.

1 TB Example

With 1 billion blocks: bitmap needs about 1 billion bits $\approx 130,000$ blocks of 1 KB. A free list with 255 addresses per block needs about 4 million list blocks.

9 Free-List Pointer Block

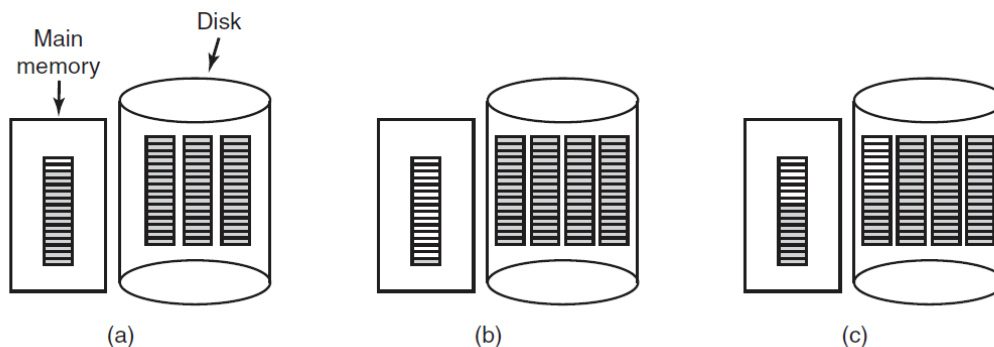


Figure 4-23. (a) An almost-full block of pointers to free disk blocks in memory and three blocks of pointers on disk. (b) Result of freeing a three-block file. (c) An alternative strategy for handling the three free blocks. The shaded entries represent pointers to free disk blocks.

- Keep one block of free-block pointers in main memory.
- File creation removes pointers from it.
- File deletion adds pointers to it.
- If it becomes empty, read another pointer block from disk.
- If it becomes full, write it to disk.

Optimization

Keep the in-memory pointer block about half full. This avoids repeated disk reads and writes when short-lived temporary files are created and deleted.

10 Disk Quotas

Definition

Disk quotas limit how many blocks and files each user may own.

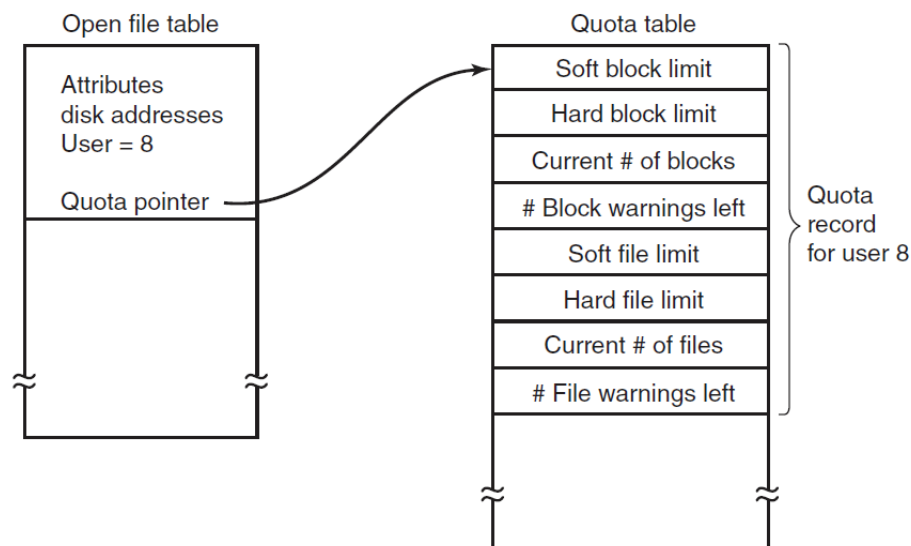


Figure 4-24. Quotas are kept track of on a per-user basis in a quota table.

Quota item	Meaning
Current blocks/files	What the user is using now.
Soft limit	Warning limit; can be exceeded temporarily.
Hard limit	Absolute limit; cannot be exceeded.
Warnings left	Count reduced when user logs in over a soft limit.

How It Works

Each open-file table entry points to the file owner’s quota record. When a block or file is added, the owner’s usage is updated and checked against soft and hard limits.

11 Final Summary

Summary

Disk-space management chooses block size, tracks free blocks, and enforces quotas. Small blocks save space; large blocks improve speed. Free space is commonly tracked by free lists or bitmaps.

File-System Backups

1 Basic Idea

Definition

File-system backup means copying file-system data to separate storage so lost, damaged, or accidentally deleted files can be restored.

Why It Matters

Replacing a computer is usually easier than reconstructing lost programs, documents, databases, records, and project files.

2 Two Main Reasons

Reason	Meaning
Recover from disaster	Disk crash, fire, flood, theft, or system failure.
Recover from stupidity	User accidentally deletes or overwrites files.

3 Backup Scope

Usually back up	Usually skip
User files	Reinstallable binaries.
Configuration files	Temporary directories.
Databases and project data	Device files such as /dev.
Important system state	Paging, hibernation, and other rebuildable internal files.

Device File Risk

Backing up special device files can be dangerous. A backup program may hang if it tries to read a device file as ordinary data.

4 Full and Incremental Backups

Type	What it saves	Trade-off
Full backup	Everything selected for backup.	Easy restore, slow backup.
Simple incremental	Files changed since the last full backup.	Faster backup, restore needs full plus incrementals.
Advanced incremental	Files changed since their last backup.	Saves time, but recovery order is more complex.

Restore Rule

Restore the latest full backup first, then apply incremental backups in order until the desired date is reached.

5 Backup Problems

Issue	Point
Compression	Saves space, but one bad tape spot may break decompression.
Active file system	Files may change while backup is running, causing inconsistency.
Snapshots	Freeze a logical view quickly; later writes use copy-on-write.
Security	Backup media must be protected like the original data.
Off-site copies	Protect against fire or local disaster, but add another site to secure.

6 Physical vs. Logical Dump

Dump type	Main idea	Limitation
Physical dump	Copy disk blocks in order, starting at block 0.	Hard to skip directories, do incrementals, or restore one file.
Logical dump	Start at selected directories and copy named files/directories.	More complex algorithm.

Practical Result

Physical dumping is simple and fast. Logical dumping is more useful for normal backups because it can restore selected files and support incremental backups.

7 Physical Dump Notes

- Do not waste time copying unused disk blocks.
- If unused blocks are skipped, the backup must record original block numbers.
- Bad blocks must be avoided, or the backup may repeatedly hit read errors.
- System-specific files, such as paging or hibernation files, may be skipped.

8 Logical Dump Example

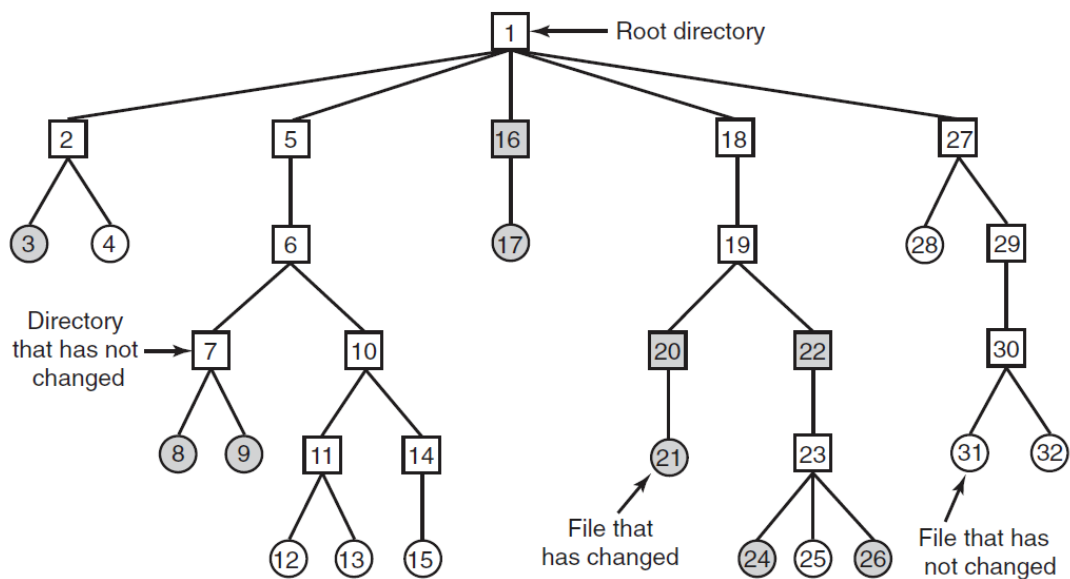


Figure 4-25. A file system to be dumped. The squares are directories and the circles are files. The shaded items have been modified since the last dump. Each directory and file is labeled by its i-node number.

Figure Meaning

Squares are directories, circles are files, and shaded items have changed since the base backup date.

Directory Rule

Dump modified files plus every directory needed to reach them, even if those directories did not change.

9 Why Unchanged Parent Directories Are Dumped

1. They allow the dump to be restored onto a fresh empty file system.
2. They allow one deleted file to be restored later with its full directory path.
3. They preserve directory attributes such as owner, mode, and timestamps.

10 Logical Dump Bitmap Algorithm

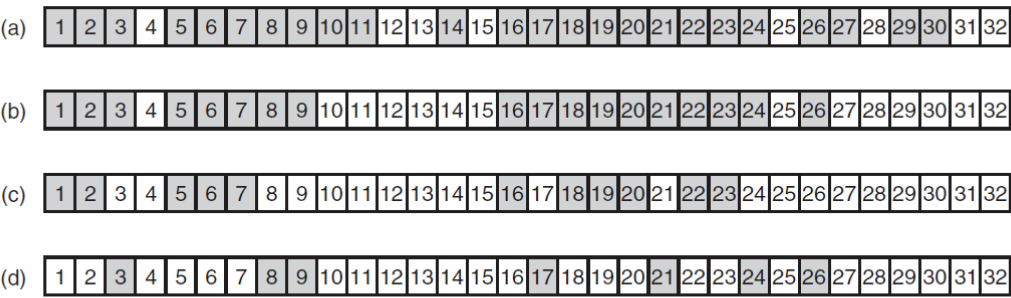


Figure 4-26. Bitmaps used by the logical dumping algorithm.

Phase	Action
1	Mark all modified files and all directories.
2	Unmark directories that contain no modified file or needed directory below them.
3	Dump marked directories in i-node order, with attributes.
4	Dump marked files in i-node order, with attributes.

11 Restoring

1. Create an empty file system.
2. Restore directories first to build the skeleton tree.
3. Restore file contents.
4. Apply later incremental dumps in sequence.
5. Rebuild the free-block list from the files that now exist.

12 Tricky Cases

Case	Required handling
Hard links	Restore file data once, then recreate all directory names pointing to it.
Sparse files	Do not dump holes; restore holes as holes, not blocks full of zeros.
Special files	Do not dump device files, named pipes, or other non-regular files as data.
Free list	Reconstruct after restore; it is not an ordinary file.

Sparse File Warning

If holes are restored as zero-filled blocks, a small sparse file can become enormous.

13 Final Summary

Summary

Backups protect against disasters and accidental deletion. Physical dumps are simple but inflexible; logical dumps are more useful because they support selected directories, incremental backups, and single-file restore.

File-System Consistency

1 Basic Idea

Definition

File-system consistency means that disk blocks, i-nodes, directories, free-space data, and link counts agree with each other.

Why It Breaks

File systems often modify blocks in memory and write them later. A crash before all writes finish can leave metadata inconsistent.

2 Critical Metadata

Metadata	Why it matters
i-node blocks	Store file ownership, link count, and disk block addresses.
Directory blocks	Map file names to i-node numbers.
Free list or bitmap	Records which disk blocks are free.
Data block pointers	Decide which file owns each disk block.

3 Consistency Checkers

Checker

A consistency checker, such as UNIX **fsck**, scans a file system after a crash and repairs metadata contradictions.

- Each partition is checked independently.
- The checker uses file-system redundancy to detect errors.
- Two main checks are block consistency and file/link consistency.

4 Block Consistency

Method

Build two counters for every disk block: one for appearances in files and one for appearances in the free list or free bitmap.

1. Read all i-nodes using raw disk access.
2. For every block used by a file, increment its “in use” counter.

3. Scan the free list or bitmap.
4. For every free block, increment its “free” counter.
5. A correct block appears exactly once: either in use or free.

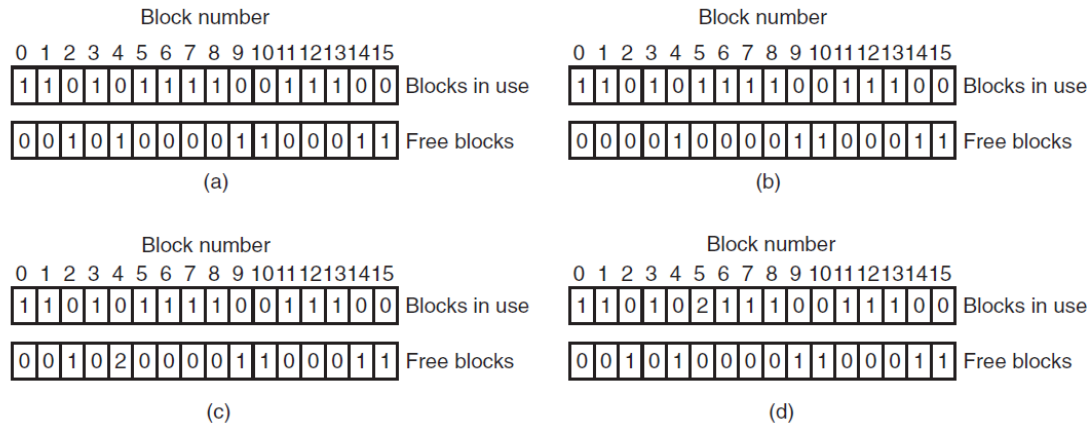


Figure 4-27. File-system states. (a) Consistent. (b) Missing block. (c) Duplicate block in free list. (d) Duplicate data block.

5 Block Error Cases

Case	Meaning	Fix
Consistent	Each block appears once, either used or free.	No repair.
Missing block	Block appears in neither table.	Add it to free list.
Duplicate free block	Block appears more than once in free list.	Rebuild free list.
Duplicate data block	Same block appears in two or more files.	Copy contents to a new block for one file, then report damage.

Worst Case

A duplicate data block is serious. If one file is deleted, the shared block may be freed even though another file still points to it.

6 Directory and Link-Count Check

Method

Count how many directory entries point to each i-node, then compare that count with the link count stored in the i-node.

- Start at the root directory.
- Recursively scan all directories.

- Increment the counter for each i-node found in a directory.
- Hard links count because they are real directory entries.
- Symbolic links do not increment the target file's count.

7 Link-Count Errors

Error	Consequence	Fix
i-node count too high	File may never be deleted, wasting disk space.	Set i-node link count to actual directory count.
i-node count too low	File may be freed while a directory still points to it.	Set i-node link count to actual directory count.

Danger

Too low is worse: deleting one name can reduce the i-node count to zero, freeing blocks that another directory entry still needs.

8 Other Suspicious Cases

Case	Why report it
Bad directory format	Directory entries should contain valid i-node numbers and names.
Invalid i-node number	Directory points outside the i-node table.
Strange permissions	Example: outsiders have more rights than the owner.
Huge directory	May indicate corruption or abnormal use.
SETUID root file in user directory	Possible security problem.

9 Protection from User Mistakes

Example

The command `rm *.o` removes object files, but `rm * .o` removes all files in the current directory and then complains that `.o` does not exist.

Recycle Bin Idea

Some systems move deleted files to a special directory first. Space is reclaimed only when files are actually deleted from that directory.

10 Final Summary

Summary

File-system checkers repair crash damage by verifying that every block is either used or free, and that directory references match i-node link counts.

File-System Performance

1 Basic Idea

Definition

File-system performance is improved by reducing slow disk accesses and making common access patterns faster.

Why Optimization Matters

Memory access may take about 10 nsec, but a disk access can include 5-10 msec of seek and rotational delay. A small disk read can be about a million times slower than memory.

2 Main Techniques

Technique	Goal
Block cache	Avoid disk reads by keeping recently used disk blocks in memory.
Block read-ahead	Fetch likely next blocks before the program asks for them.
Reduced arm motion	Place related blocks close together on disk.

3 Block Cache

Buffer Cache

A **block cache** or **buffer cache** keeps disk blocks in memory so repeated reads can be served without disk I/O.

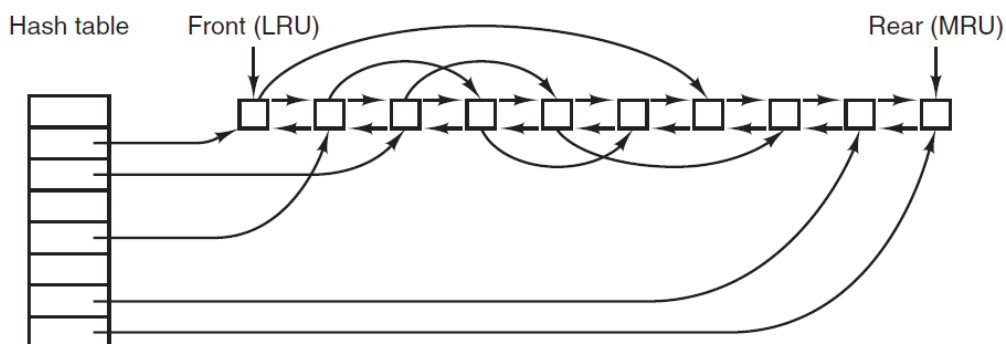


Figure 4-28. The buffer cache data structures.

- On every read, the file system first checks the cache.
- Cache hit: return the block from memory.

- Cache miss: read the block from disk, place it in cache, then return it.
- A hash table maps (device, disk block) to cache entries.
- Collision chains handle multiple blocks with the same hash value.

4 LRU Cache List

LRU

Least recently used replacement keeps old blocks near the front and recently used blocks near the rear.

Action	Result
Block is referenced	Move it to the rear, the MRU end.
Cache is full	Reuse a block near the front, the LRU end.
Dirty block is removed	Write it to disk first.

5 Why Pure LRU Is Not Enough

Metadata Risk

If a dirty i-node, directory block, or indirect block stays in cache too long, a crash can leave the file system inconsistent.

Question	Cache policy effect
Needed again soon?	Keep likely reusable blocks longer.
Critical for consistency?	Write dirty metadata blocks quickly.
Plain data block?	Can usually wait longer than metadata.
Partly full output block?	Keep near MRU because more writes may arrive soon.

6 Write Policy

Policy	Meaning	Trade-off
Periodic write-back	Dirty blocks stay in cache and are flushed by sync .	Efficient, but recent data may be lost in a crash.
Write-through	Every modified block is written to disk immediately.	Safer for removable media, but more disk I/O.

UNIX Example

A background process can call **sync** about every 30 seconds, limiting typical crash loss to recent changes.

7 Unified Cache

Single Cache

Some systems combine the buffer cache and page cache, especially when memory-mapped files are supported.

Mapped file pages and cached file blocks both represent file data in memory, so one shared cache avoids duplicate copies.

8 Block Read-Ahead

Read-Ahead

When block k is read, the file system may also start reading block $k + 1$, assuming sequential access.

Access pattern	Read-ahead result
Sequential	Helps; next block may already be in cache.
Random	Hurts; wastes bandwidth and cache space.

Simple Detection

Keep a per-file mode bit: assume sequential at first, clear it after a seek, and set it again if sequential reads resume.

9 Reducing Disk-Arm Motion

Locality

Place blocks likely to be accessed together near each other, preferably in the same cylinder or cylinder group.

- Bitmap allocation makes it easier to choose a free block near the previous block.
- Free lists make close allocation harder unless they track runs of consecutive blocks.
- Clustering allocates groups of consecutive blocks to reduce seeks.
- Rotational placement can also place consecutive blocks favorably within a cylinder.

10 I-Node Placement

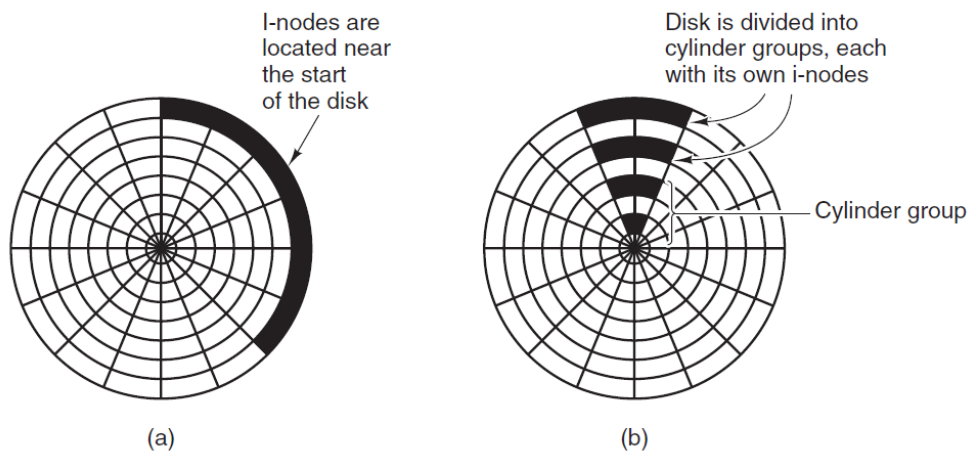


Figure 4-29. (a) I-nodes placed at the start of the disk. (b) Disk divided into cylinder groups, each with its own blocks and i-nodes.

Placement	Effect
All i-nodes at disk start	Short files need one seek for i-node and another for data; average seek is large.
I-nodes near disk middle	Cuts average i-node-to-data seek distance.
Cylinder groups	Each group has its own i-nodes, blocks, and free list; new file data is placed near its i-node.

11 SSDs

Different Device

SSDs have no disk arm or rotational delay, so random access is much closer to sequential access. However, blocks have limited write endurance, so wear leveling becomes important.

12 Final Summary

Summary

File systems improve performance by caching disk blocks, reading ahead for sequential files, and placing related blocks near each other. Metadata must be written carefully because performance optimizations must not destroy consistency.